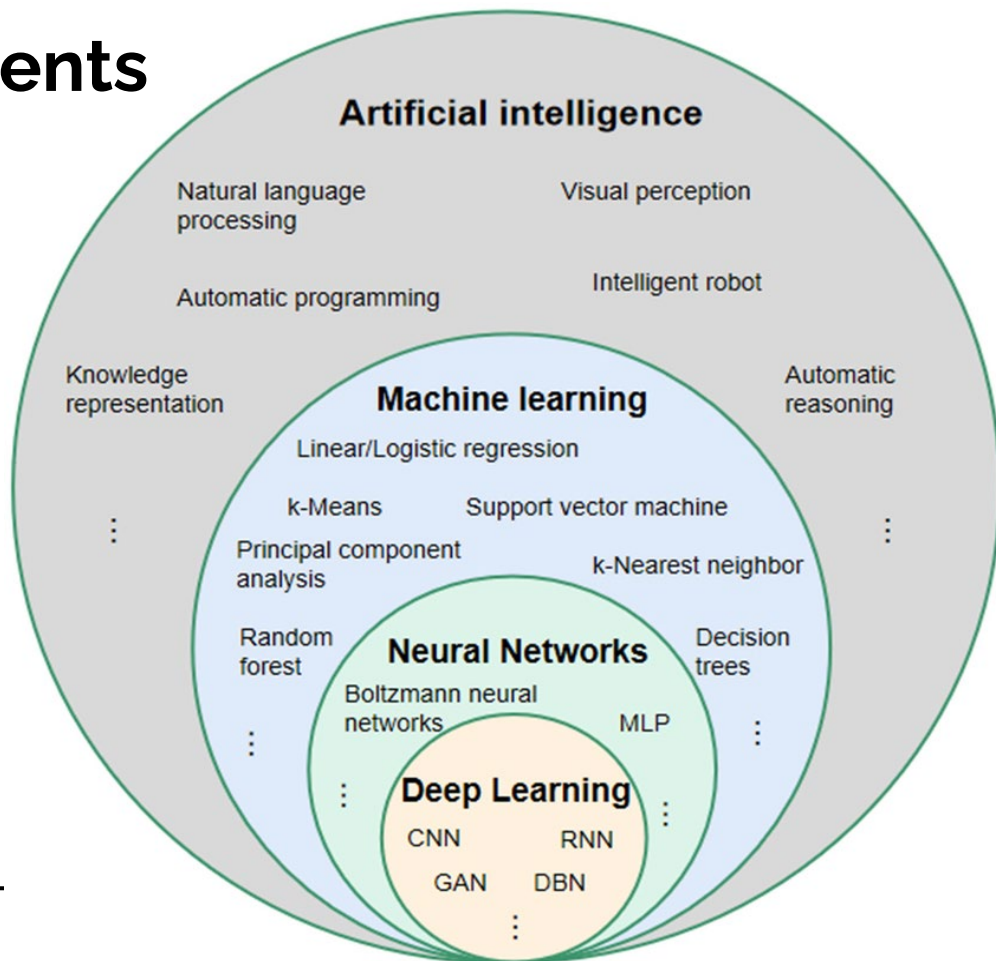

Machine Learning for Neuroscience

Prediction and Learning



Contents



Don't be scared of data

The diagram illustrates an $m \times n$ data matrix. A vertical arrow on the left is labeled 'SAMPLES' and points downwards. A horizontal arrow at the top is labeled 'VARIABLES' and points to the right. The matrix is a grid of colored cells, each containing a '#' symbol. The columns are labeled $V_1, V_2, V_3, \dots, V_M$ at the top. The rows are labeled $S_1, S_2, S_3, S_4, S_5, \dots, S_N$ on the left. The cells are colored in a repeating pattern of six colors: green, yellow, grey, pink, orange, and blue.

	V_1	V_2	V_3	$\dots$	V_M
S_1	#	#	#	#	#
S_2	#	#	#	#	#
S_3	#	#	#	#	#
S_4	#	#	#	#	#
S_5	#	#	#	#	#
$\dots$	#	#	#	#	#
S_N	#	#	#	#	#

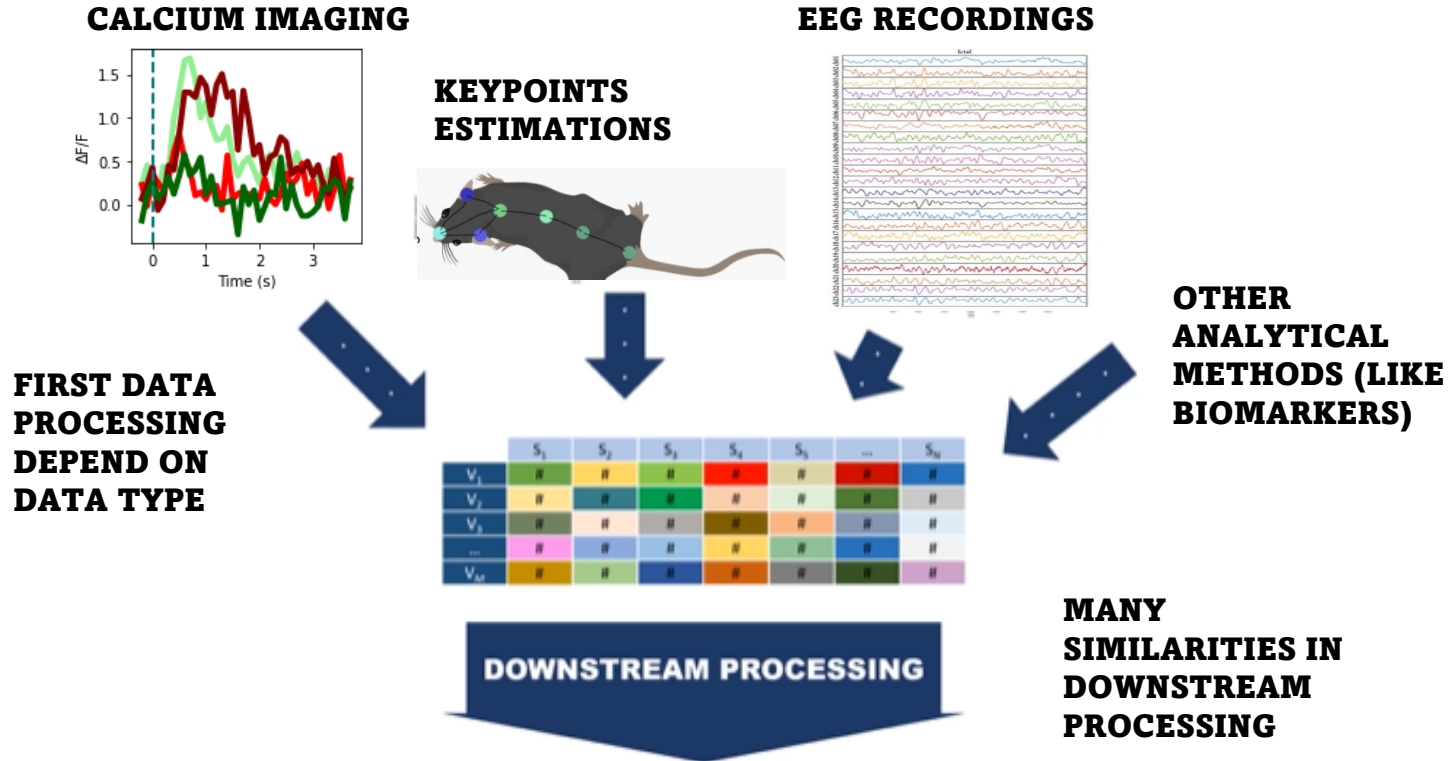
This is an $m \times n$ data matrix, or data frame

A **variable** (or label) can be the amount of Ca at a specific time point, a keypoint, a gene, a protein...

A **sample** could be a person, a mouse, a neuron...

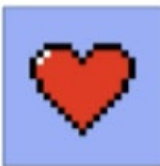
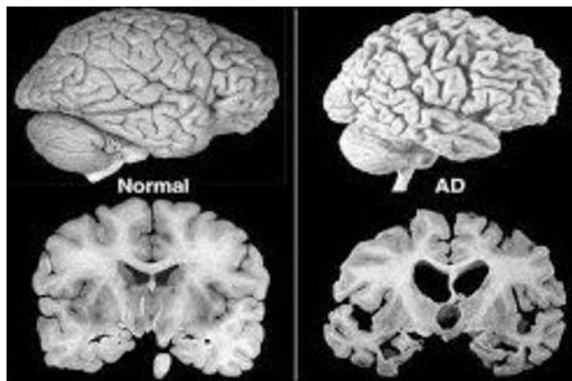
The number in each cell is the value of one variable in a specific sample

The datasets take mostly the same form



But not all data takes the same form

IMAGES



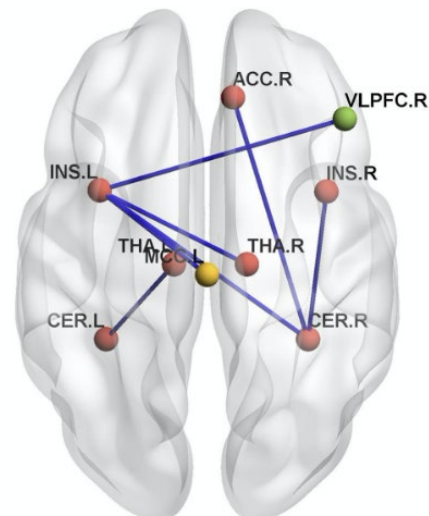
STRINGS

Granular cell tumour is a tumour that can develop on any skin or mucosal surface, but occurs on the tongue 40% of the time.

```
CAACACGTAAAGGGTGT
CGAGACACAGCTGCTCC
TGAGTACACAAAACGGT
CAGCTACGAACTCATTT
```

```
MLGTGPAATAATTSSNVSVLQQFASGLKS
RNEETRAKAAKELQHYVTMELREMSQEEER
FYDQLNHHIFELVSSSDANERKGGILAIAS
LIGVEGGNATRIGRFANYLRNLLPSNDPVM
EMASKAIGRLAMAGDTFAEYVEFEVRALEW
```

CONNECTIONS



Types of learning

Unsupervised

Algorithms learn patterns exclusively from unlabelled

Used for specific algorithms such as clustering, dimensionality reduction, and association

Supervised

You have data that is labelled

Typically used in most ML problems

Requires gathering of large datasets

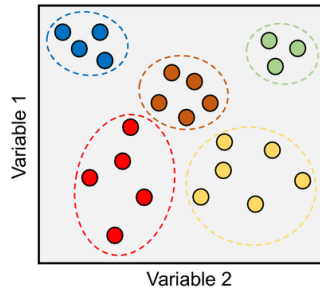
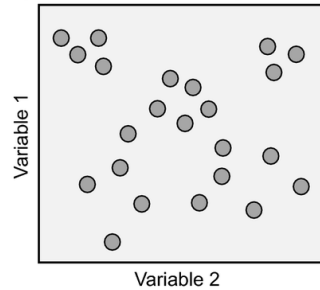
Types of learning

Unsupervised

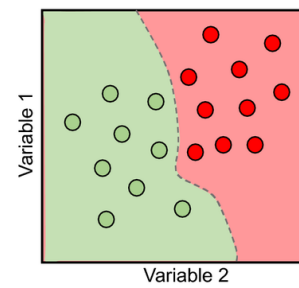
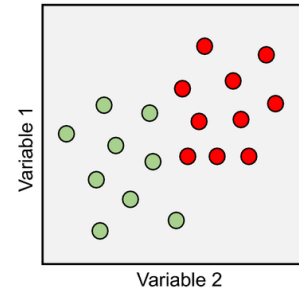
Explore relationships among samples and variables

Look for **patterns** and **clusters**

a) Unsupervised learning



b) Supervised learning



Supervised

Build **boundaries** between samples with known labels

Often called **MACHINE LEARNING**

Exploratory data analysis questions

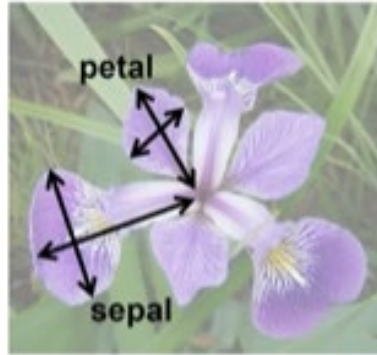
- What is the relationship between samples?
- What is the relationship between the measured variables?
- Are there any outlying samples or variables?
- How much variation is there between samples of the same type?
- Is there any point in analysing the data further, e.g. with machine learning?

Fisher's Multivariate Iris Dataset

Iris setosa



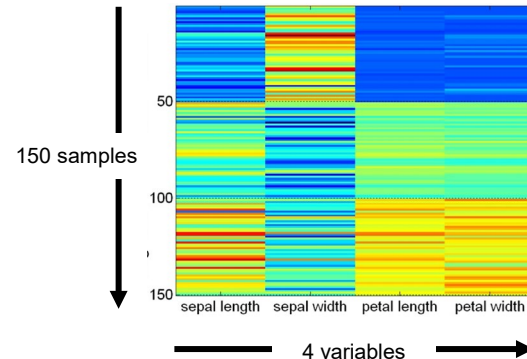
Iris versicolor



Iris virginica



4 measurements
taken from 150
individual iris flowers

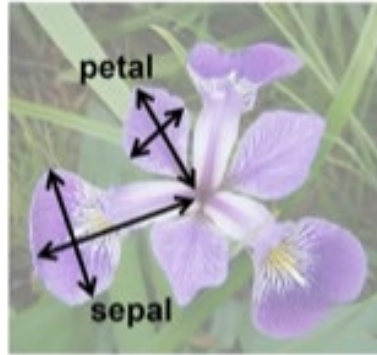


Fisher's Multivariate Iris Dataset

Iris setosa



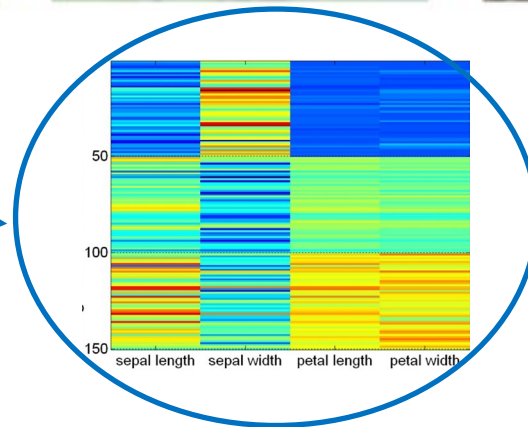
Iris versicolor



Iris virginica



Data matrix

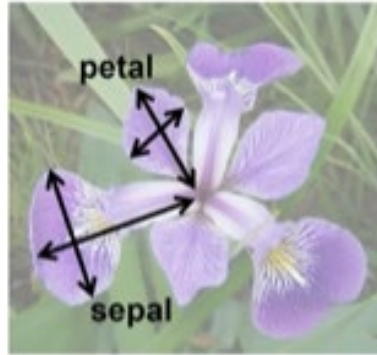


Fisher's Multivariate Iris Dataset

Iris sentosa



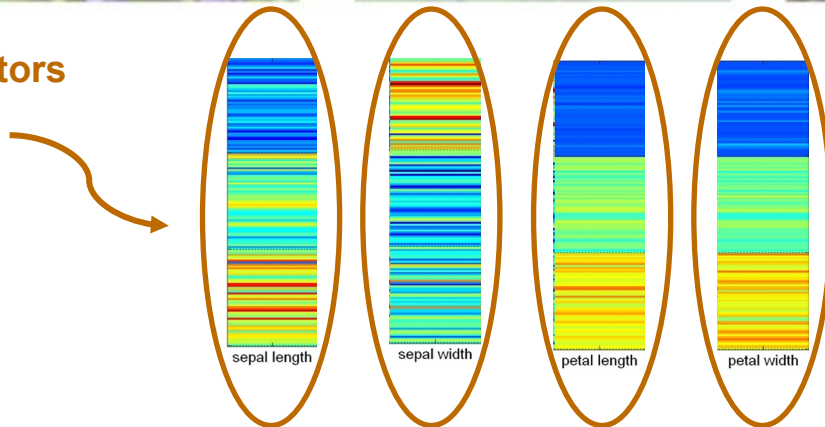
Iris versicolor



Iris virginica



Class (or label) vectors



Fisher's Multivariate Iris Dataset

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

150 rows × 5 columns

What do you do?

You start exploring

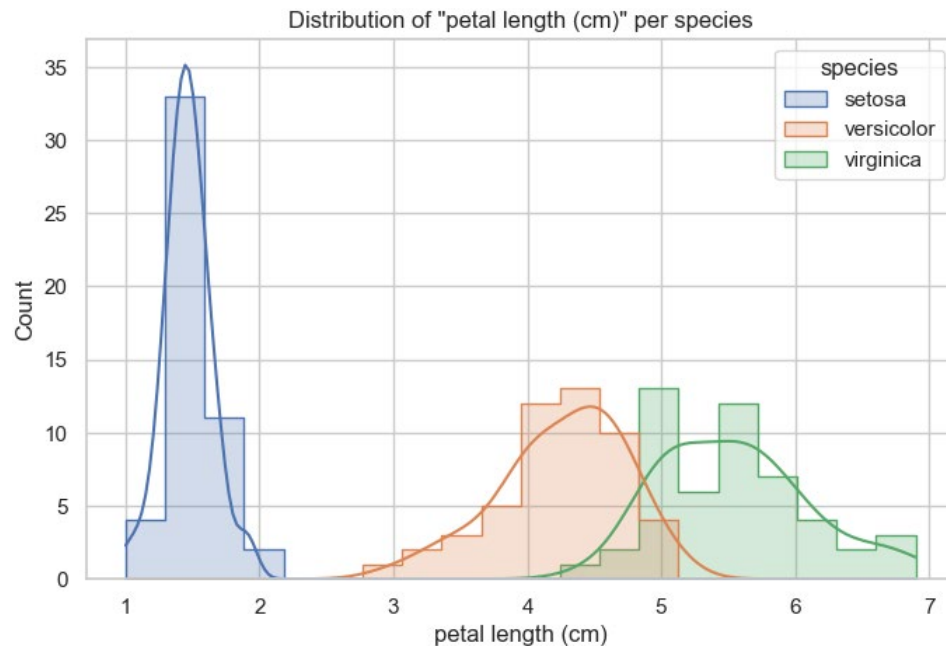
Univariate exploratory analysis

We could compare each variable in turn.

For example, here is the distribution of petal lengths

	species	petal length (cm)
0	setosa	1.4
1	setosa	1.4
2	setosa	1.3
3	setosa	1.5
4	setosa	1.4
...	...	...
145	virginica	5.2
146	virginica	5.0
147	virginica	5.2
148	virginica	5.4
149	virginica	5.1

150 rows × 2 columns



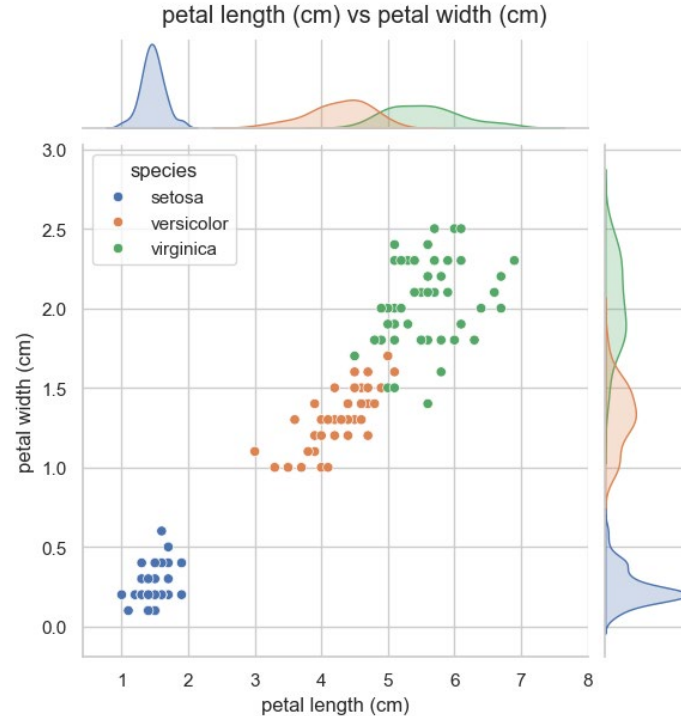
Bivariate exploratory analysis

We can plot pairs of variables

Are we starting to see something?

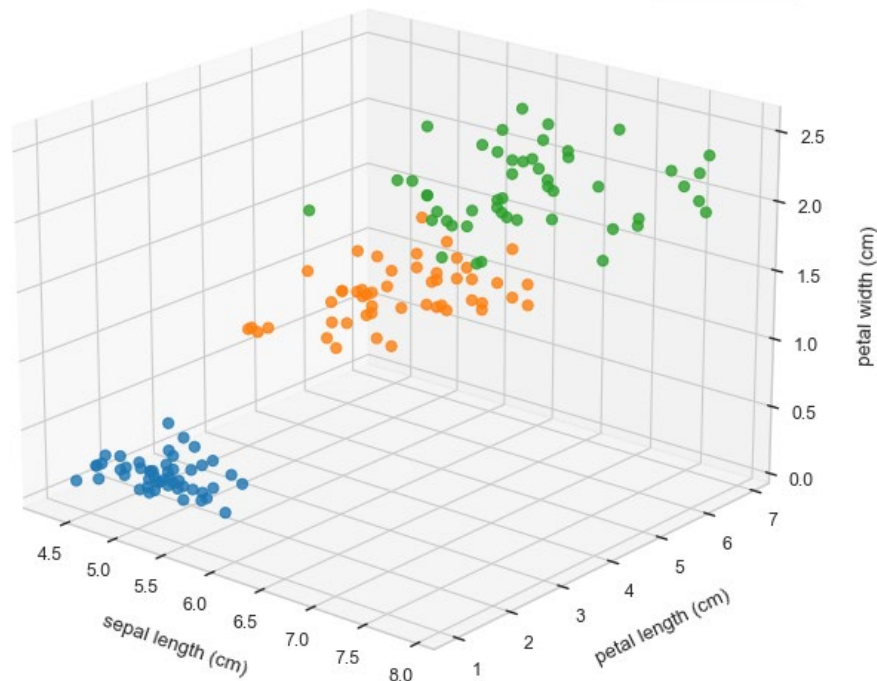
	species	petal length (cm)	petal width (cm)
0	setosa	1.4	0.2
1	setosa	1.4	0.2
2	setosa	1.3	0.2
3	setosa	1.5	0.2
4	setosa	1.4	0.2
...	...	...	...
145	virginica	5.2	2.3
146	virginica	5.0	1.9
147	virginica	5.2	2.0
148	virginica	5.4	2.3
149	virginica	5.1	1.8

150 rows × 3 columns



3D scatter plot analysis

We can plot 3 variables in 3D



	species	petal length (cm)	petal width (cm)	sepal length (cm)
0	setosa	1.4	0.2	5.1
1	setosa	1.4	0.2	4.9
2	setosa	1.3	0.2	4.7
3	setosa	1.5	0.2	4.6
4	setosa	1.4	0.2	5.0
...	...	...	...	...
145	virginica	5.2	2.3	6.7
146	virginica	5.0	1.9	6.3
147	virginica	5.2	2.0	6.5
148	virginica	5.4	2.3	6.2
149	virginica	5.1	1.8	5.9

150 rows × 4 columns

4D multivariate analysis

	species	petal length (cm)	petal width (cm)	sepal length (cm)	sepal width (cm)
0	setosa	1.4	0.2	5.1	3.5
1	setosa	1.4	0.2	4.9	3.0
2	setosa	1.3	0.2	4.7	3.2
3	setosa	1.5	0.2	4.6	3.1
4	setosa	1.4	0.2	5.0	3.6
...	...	...	...	...	...
145	virginica	5.2	2.3	6.7	3.0
146	virginica	5.0	1.9	6.3	2.5
147	virginica	5.2	2.0	6.5	3.0
148	virginica	5.4	2.3	6.2	3.4
149	virginica	5.1	1.8	5.9	3.0

150 rows × 5 columns

**How can we visualise datasets that
have more than 3 variables?**

We can't

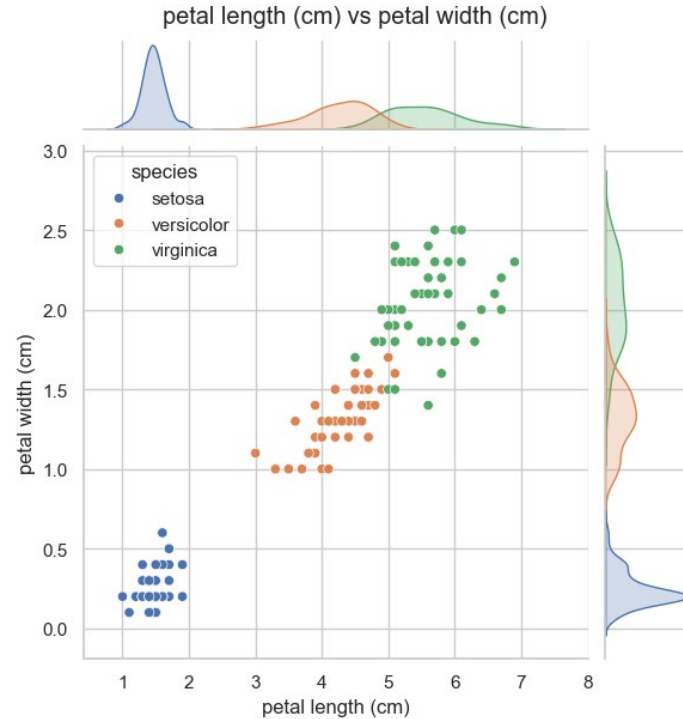


Dimensionality reduction

Principal Component Analysis (PCA)

Let's take a step back to the bivariate exploratory analysis

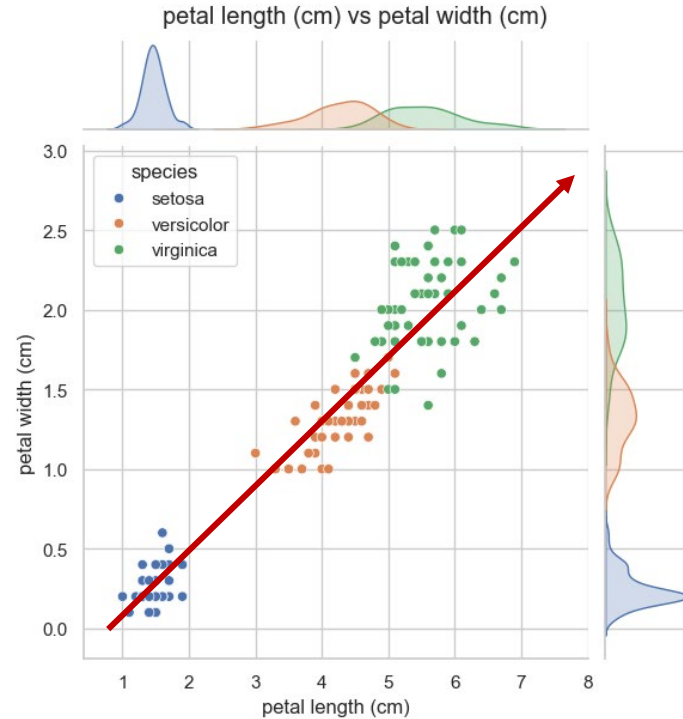
What can we see about the relationship between petal length and petal width?



Principal Component Analysis (PCA)

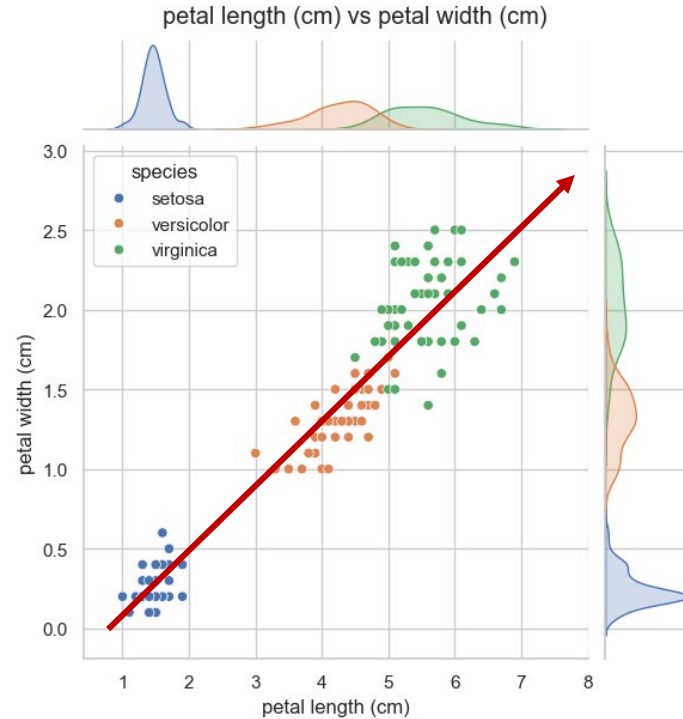
Let's take a step back to the bivariate exploratory analysis

What can we see about the relationship between petal length and petal width?



Principal Component Analysis (PCA)

- Petal length and petal width move together. They are **correlated**
- Correlated measurements carry **overlapping information**
- Like getting temperature in both Celsius and Fahrenheit. Two columns, one real dimension.



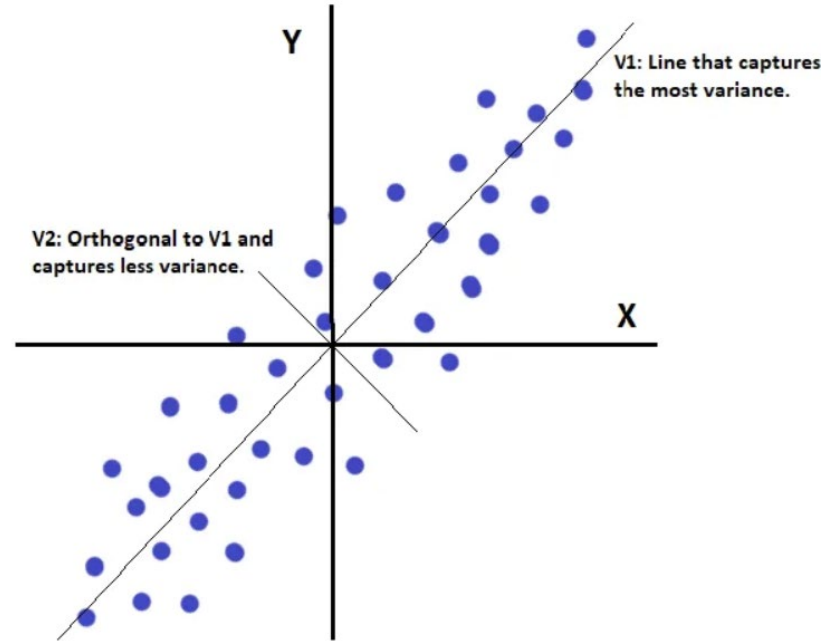
Principal component analysis

Principal components are new variables that are constructed as **linear combinations or mixtures of the initial variables**.

In simple terms it is reducing the number of variables from a data set

This method helps in identifying the directions (or components) in which the data varies the most

Like looking at a 2D shadow of a 3D object



Principal Component Analysis (PCA)

- Principal components are pulled out in order of the amount of variance they contribute to the data matrix, which is ultimately related to information content, while the noise tends to be relegated to lower PCs.

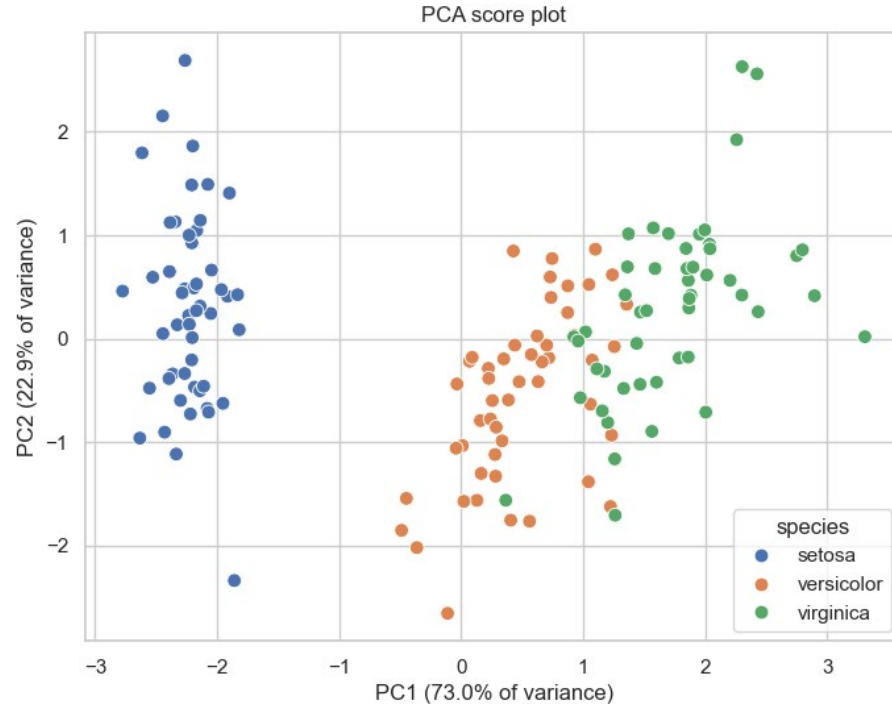
```
PC1: 72.96% of the variance  
PC2: 22.85% of the variance  
PC3: 3.67% of the variance  
PC4: 0.52% of the variance
```

```
First two components together: 95.81%
```

Principal Component Analysis (PCA)

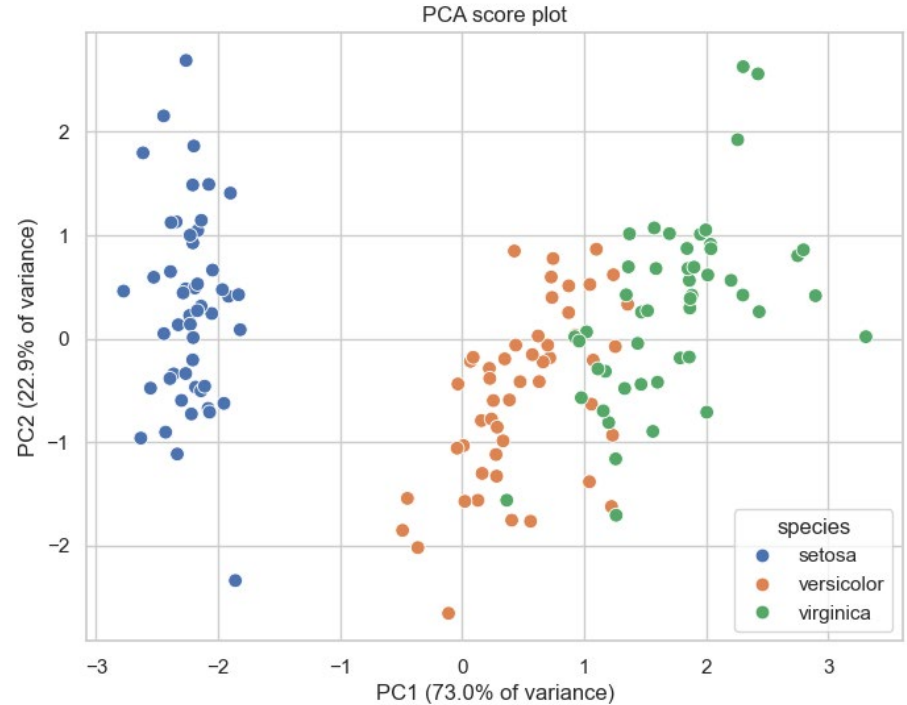
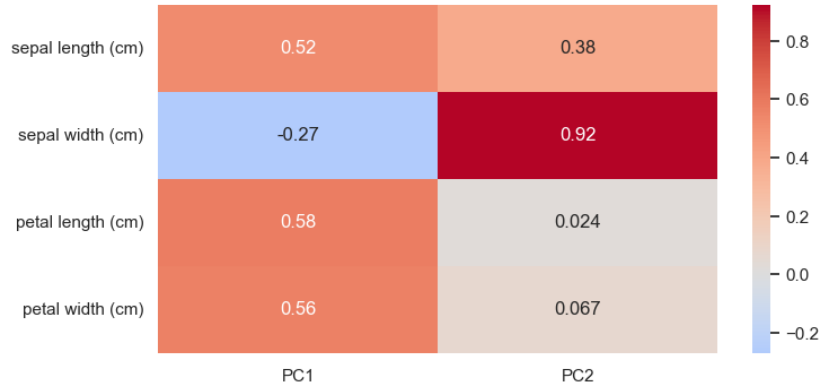
PC1: 72.96% of the variance
PC2: 22.85% of the variance
PC3: 3.67% of the variance
PC4: 0.52% of the variance

First two components together: 95.81%



PCA loadings

PCA loadings (how each feature contributes)

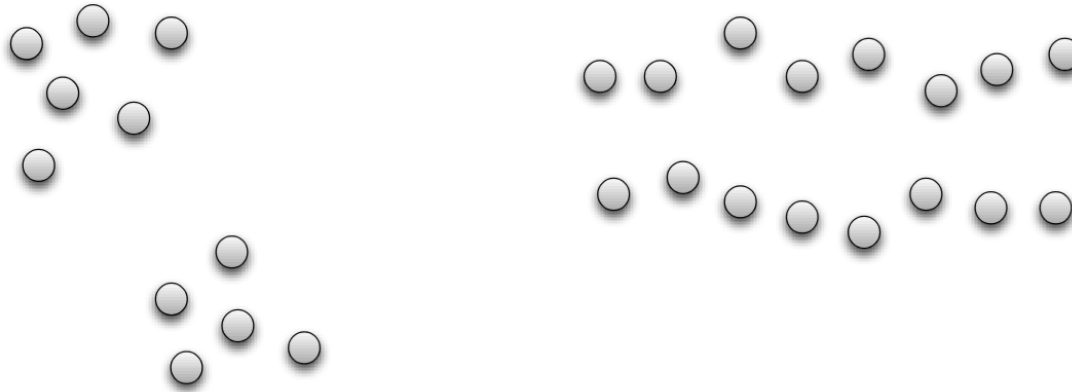


Limitations of PCA

- PCA is a powerful method of looking at the distribution of samples within a data set.
- But, PCA scores plots become a bit confusing when:
 - there are a very large number of samples
 - the data cannot be captured in three or fewer dimensions
- It's also difficult to define clusters in PCA.

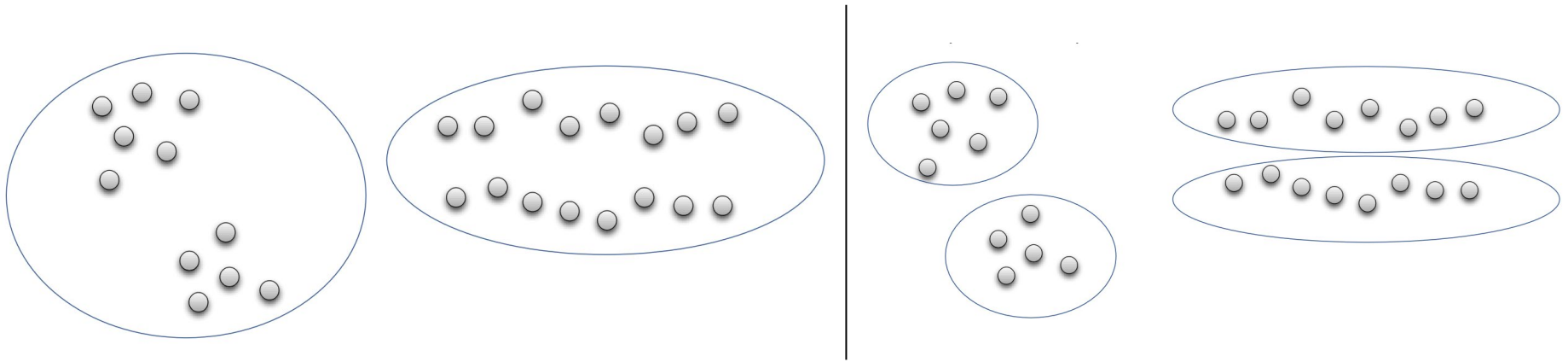
Clustering

- Basic idea: group together similar instances



Clustering

- Basic idea: group together similar instances



-> dependent on the measure of similarity (or distance) between “points” to be clustered

Clustering

- Basic idea: group together similar instances

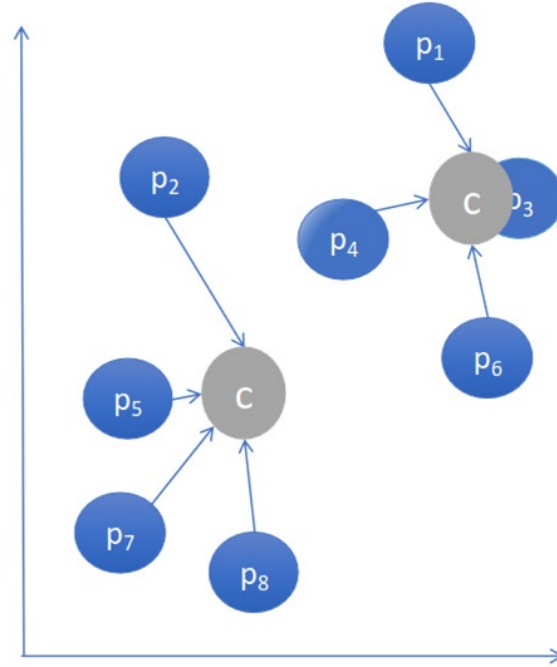
Types of clustering:

- Hierarchical clustering (dendrograms)
- Centroid-based clustering 
- Density-based clustering
- Distribution-based clustering

-> dependent on the measure of similarity (or distance) between “points” to be clustered

K-means Algorithm

- **Unsupervised:** No labels or predefined categories.
- **Clusters:** Groups of points that are more similar to each other than to those in other clusters.
- **Centroid:** The "centre" of a cluster, calculated as the mean of all points in the cluster.



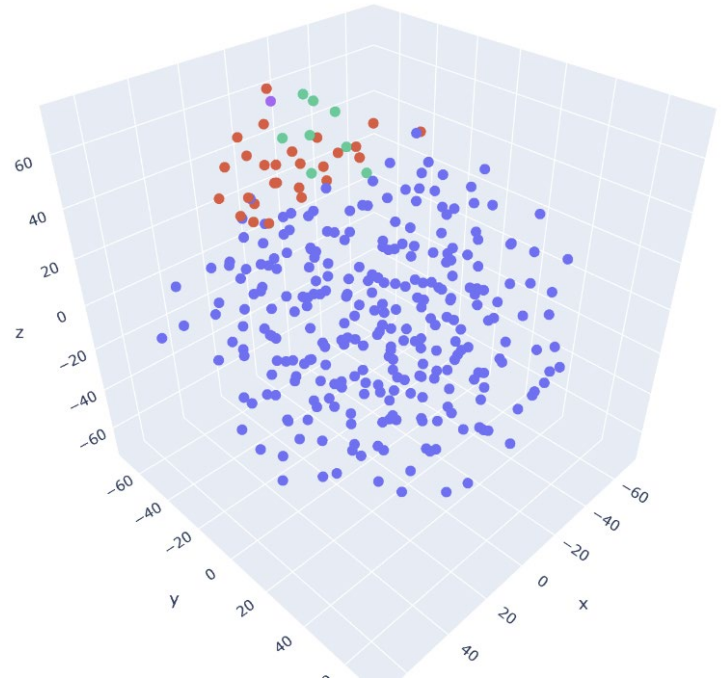
K-means Algorithm

In neuroscience, this is ideal for:

Grouping neurons with similar activity patterns;

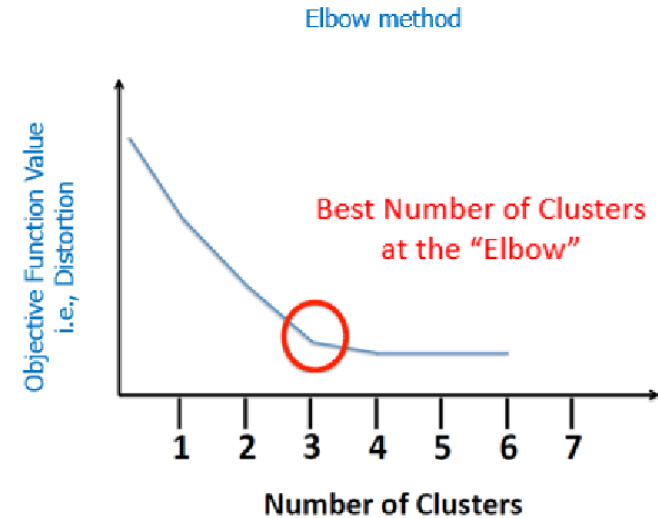
Identifying states in brain imaging data;

Segmenting behavioral responses in trial data



K-Means

How many clusters to choose?

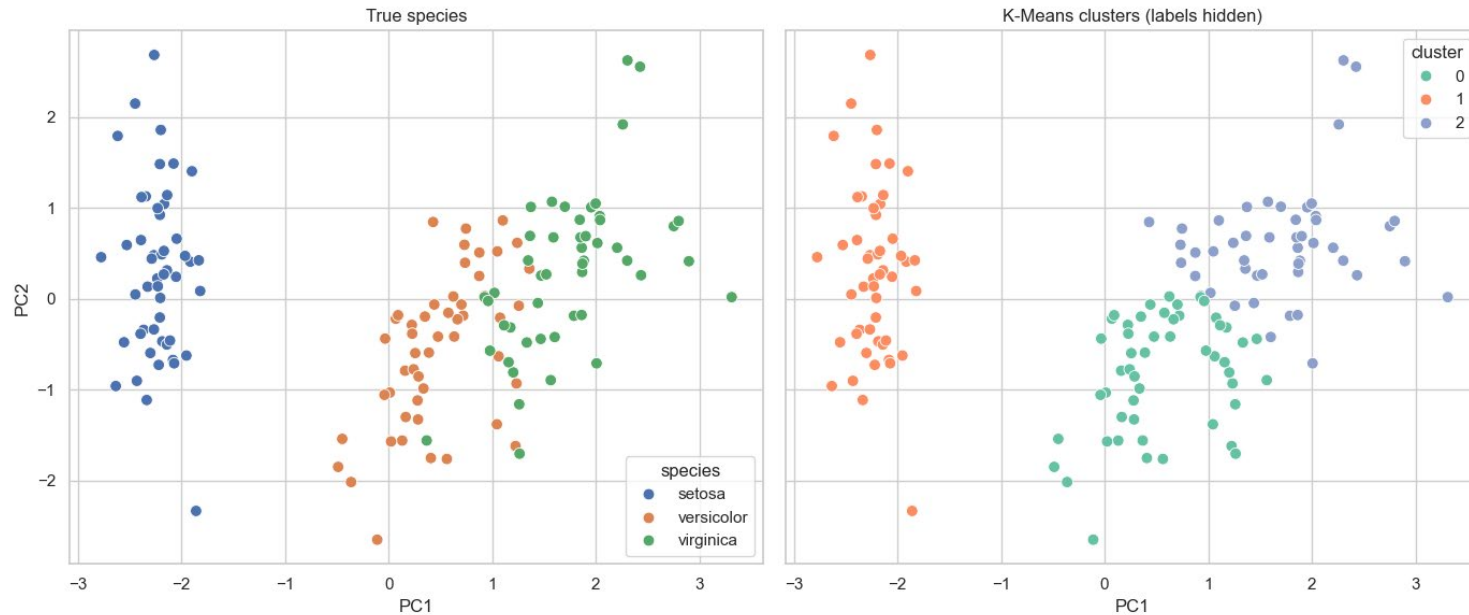


K-Means



petal length (cm)	petal width (cm)	sepal length (cm)	sepal width (cm)
1.4	0.2	5.1	3.5
1.4	0.2	4.9	3.0
1.3	0.2	4.7	3.2
1.5	0.2	4.6	3.1
1.4	0.2	5.0	3.6
...	...	...	...
5.2	2.3	6.7	3.0
5.0	1.9	6.3	2.5
5.2	2.0	6.5	3.0
5.4	2.3	6.2	3.4
5.1	1.8	5.9	3.0

K-Means



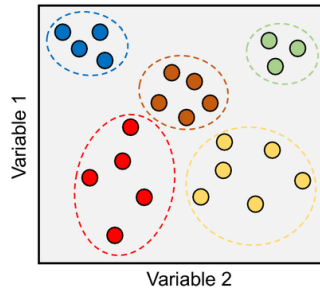
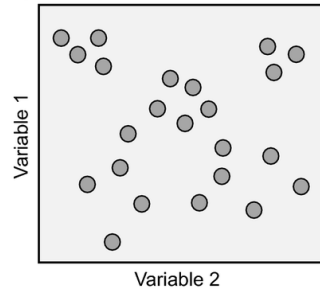
Types of learning

Unsupervised

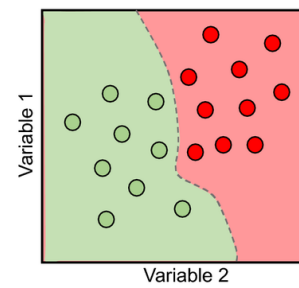
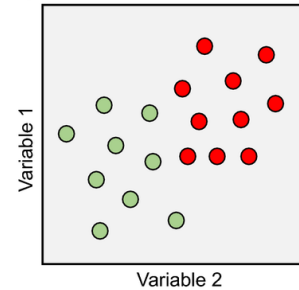
Explore relationships among samples and variables

Look for **patterns** and **clusters**

a) Unsupervised learning



b) Supervised learning



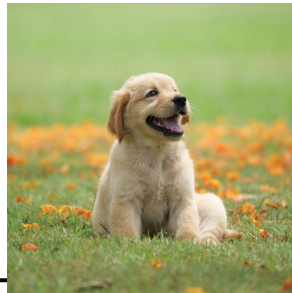
Supervised

Build **boundaries** between samples with known labels

Often called **MACHINE LEARNING**

Supervised example

We have our data (known as X)

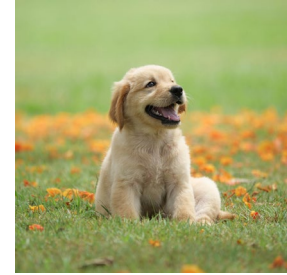


Supervised example

We have our labels (known as y)



$y = \text{Class} = \text{cat}$

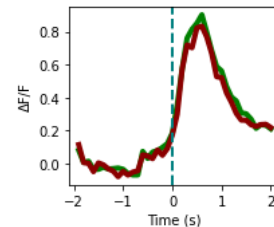
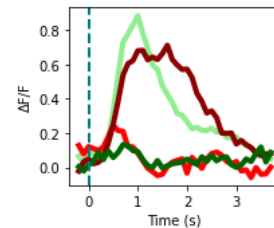
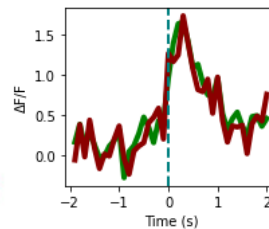
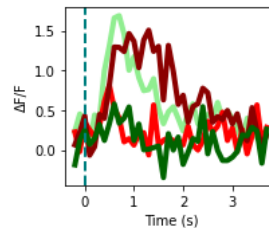
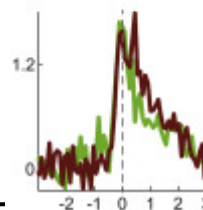
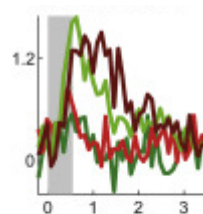
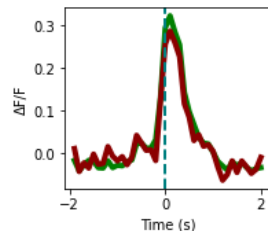
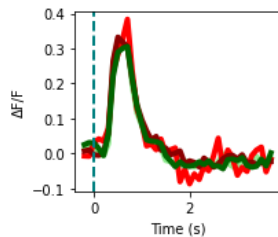
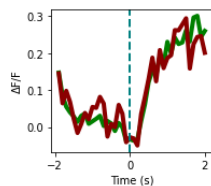
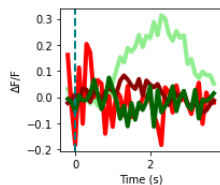
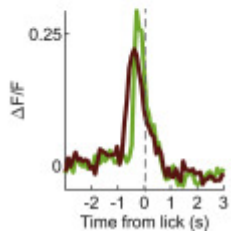
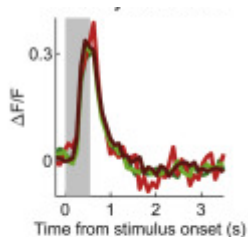


$y = \text{Class} = \text{dog}$



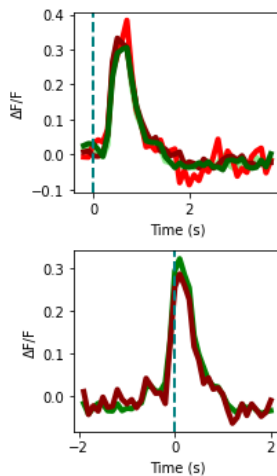
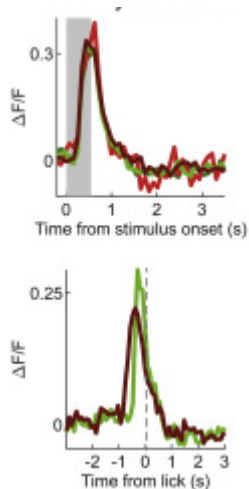
Supervised example

Our data (X)

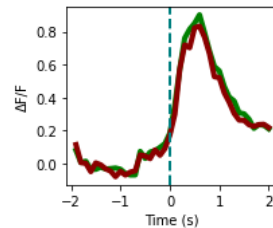
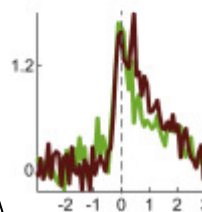
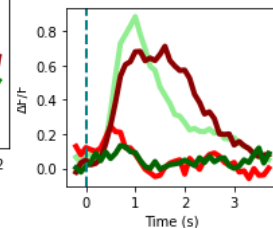
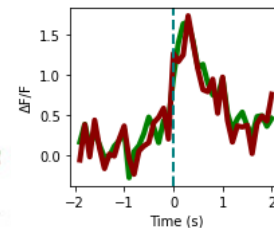
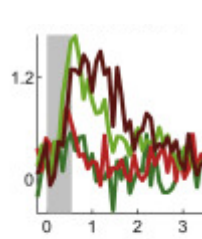
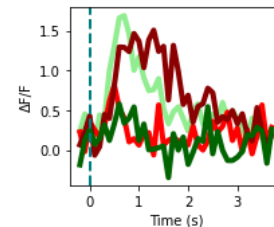


Supervised example

Our labels (y)



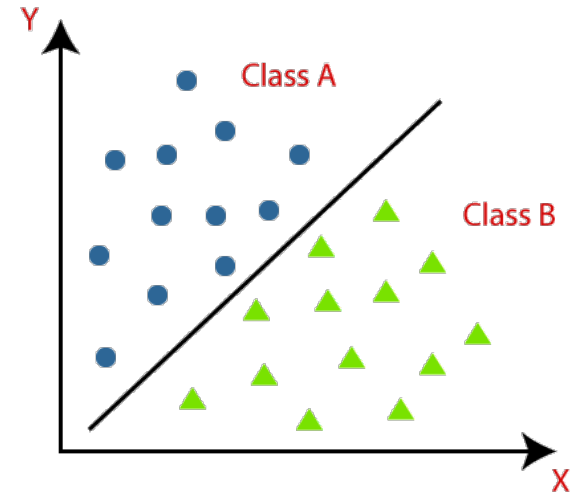
y = Class=sensory cue/ touch



y = Class= decision / action

Overview of Classification

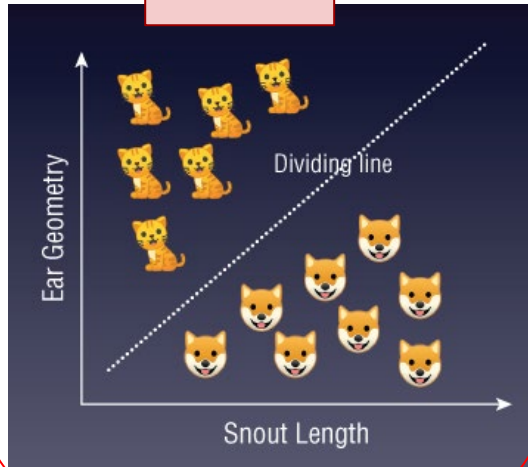
Classification is a machine learning technique used to categorize data into distinct groups or classes. It helps us assign labels to new data points based on patterns learned from past data.



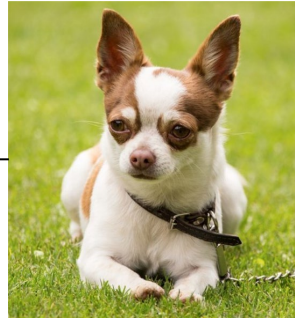
Binary Classification

Goal: find decision boundary (*dividing line between classes*)

SVM



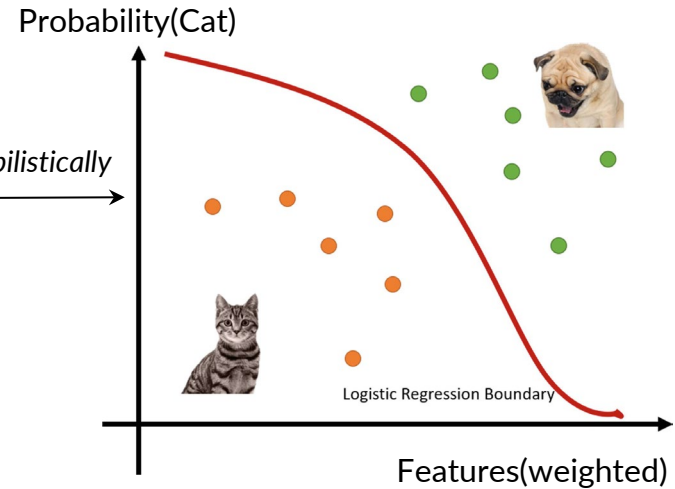
Geometrically



Cat or Dog?

Probabilistically

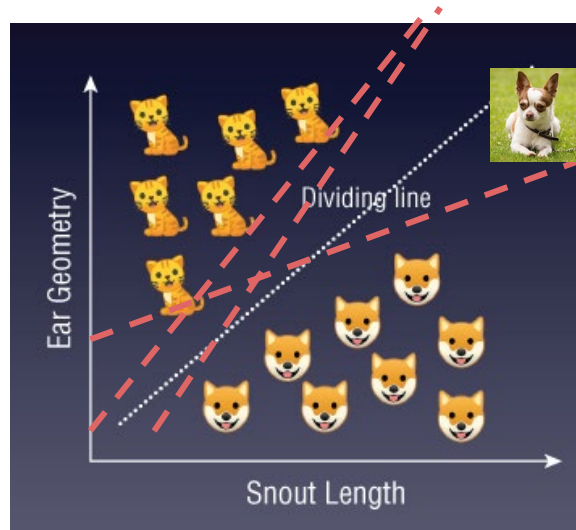
Logistic Regression



Binary Classification - SVM

Goal: find **lower-dimensional** decision boundary - hyperplanes

-> Separates classes in the feature space



-> Cat X

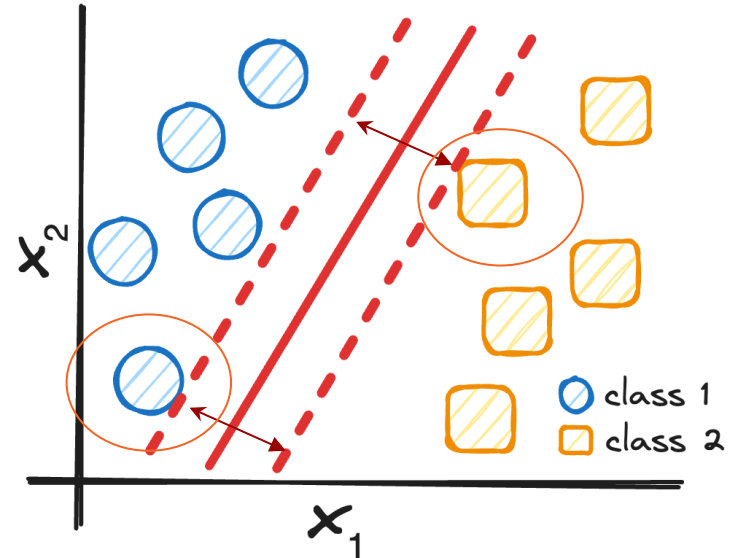
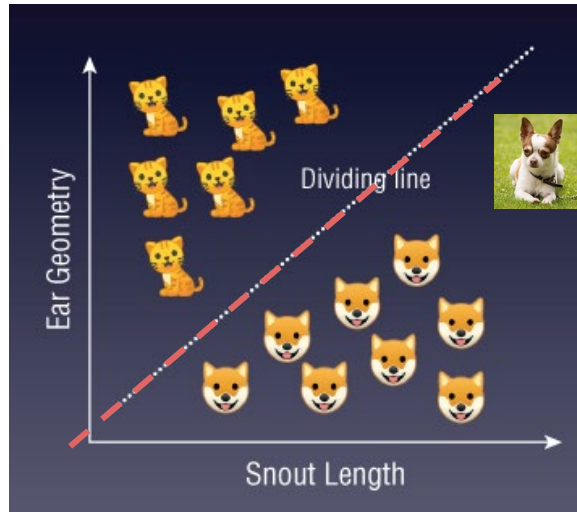


Cat or Dog?

Support Vector Machine (SVM)

Goal: find **hyperplane** that maximizes the **margin** between the **support vectors**

$w \cdot x + b = 0$ distance nearest points of each class



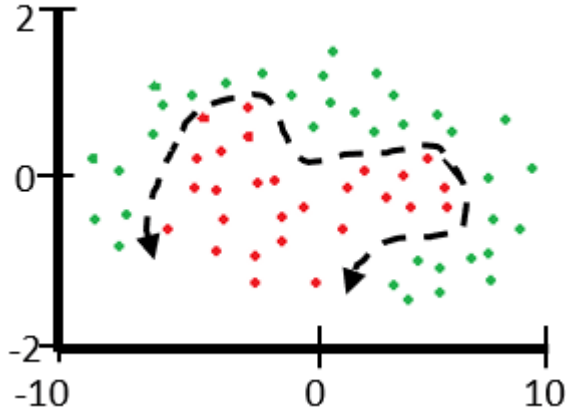
What if we have non-linearly separable data?

-> **no optimal hyperplane possible!**

Standard SVM fails when data is not linearly separable.

Solutions:

- **Soft Margin SVM:** allows misclassification by changing constrain function
- **Kernel Trick:** maps data into higher dimensional space

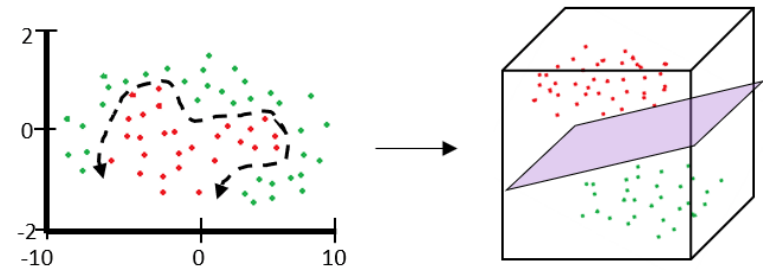
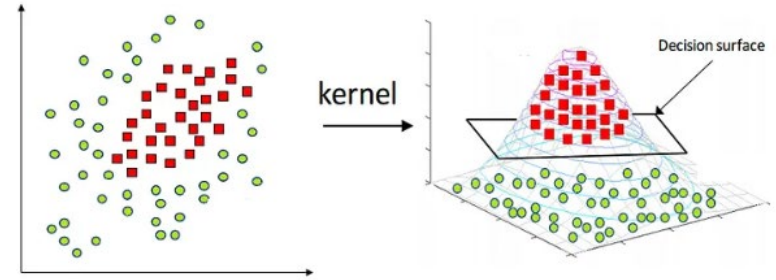


Kernel Functions

Use a kernel function to transform data into a higher-dimensional space before finding hyperplane.

Common Kernel Functions:

- **Linear:** no transformation, standard SVM
- **Polynomial:** curved decision boundaries
- **RBF (Gaussian), Sigmoid:** non-linear data, complex patterns



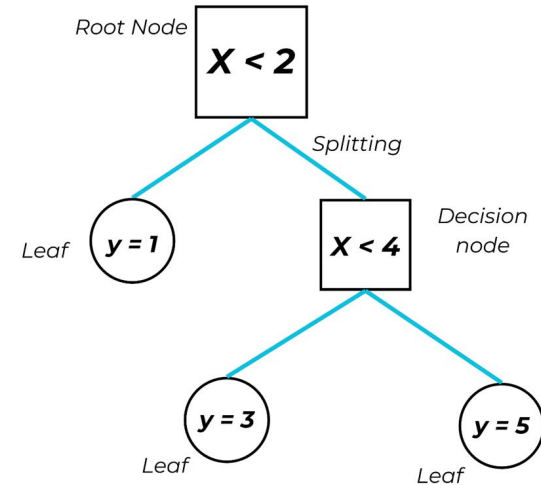
More complex classification problems

Random Forest

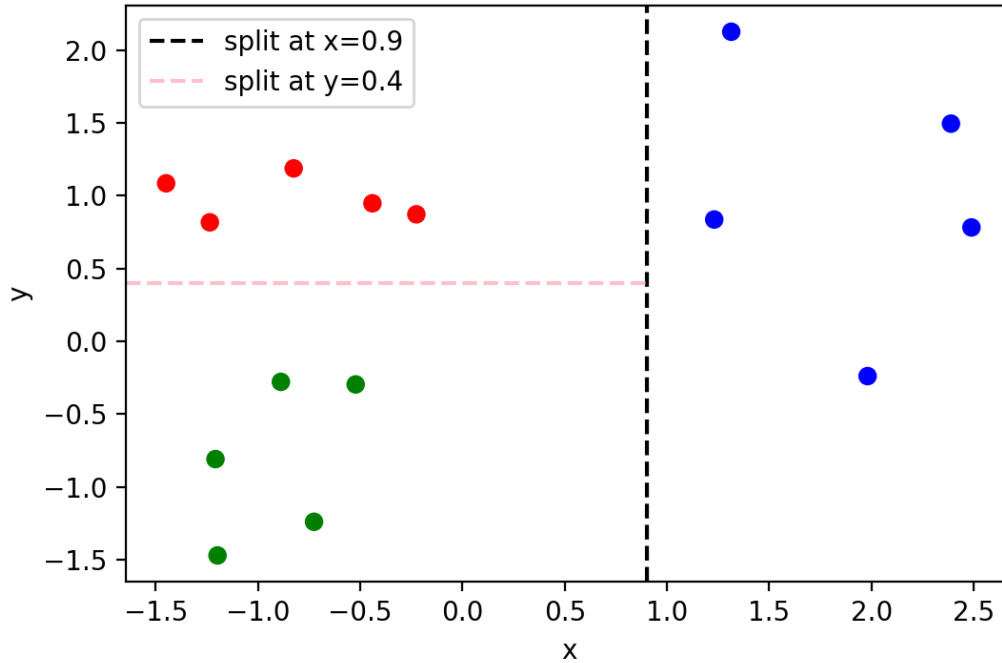
Decision Trees

Learns patterns in data by repeatedly splitting based on features, leading to a decision boundary that classifies the input data.

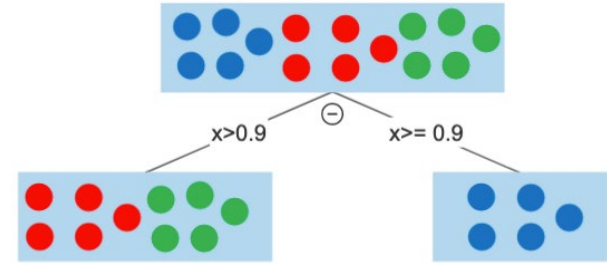
- **Root Node:** Starting point
- **Decision Nodes:** Split data based on features
- **Leaf Nodes:** Final nodes with predictions
- **Output:** Class label at the leaf node



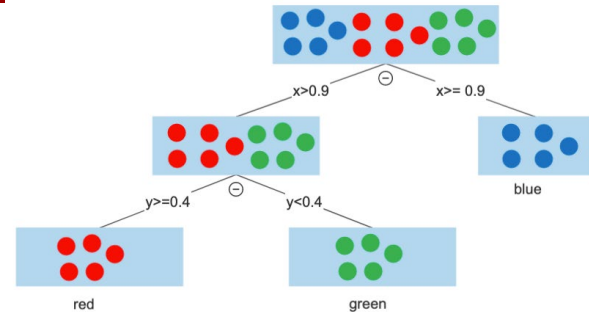
Example



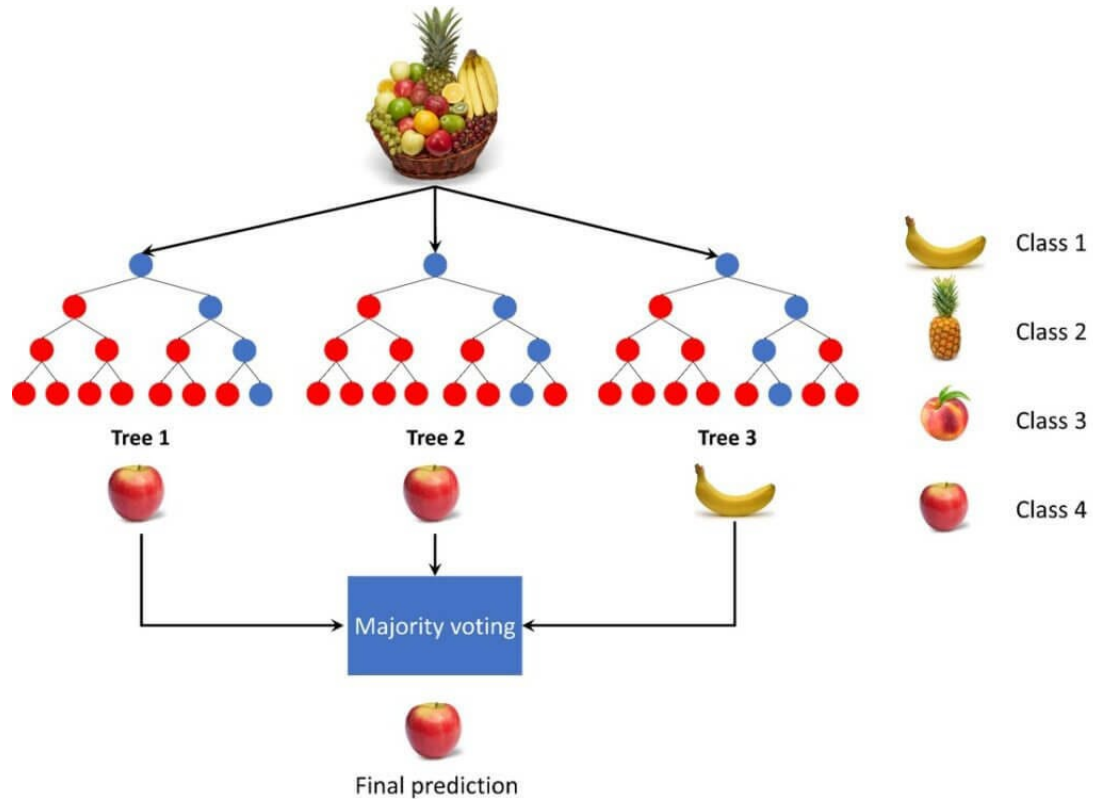
1.



2.



Example





DECISION TREES

Random Forest: Ensemble of decision trees

DECIS



DECISION TREES



DECISION TREES



DECISION TREES



DECISION TREES

- A collection ("forest") of Decision Trees.
- Each tree is trained on a **random subset of the data** and a **random subset of features**.
- The forest **aggregates the output** of all trees
- For classification: takes the **majority vote**.

Splitting this data

Data splitting allows us to build robust models that perform well on training data and generalise effectively to new, unseen data.

Training data

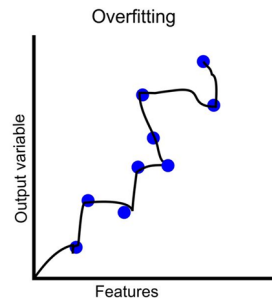
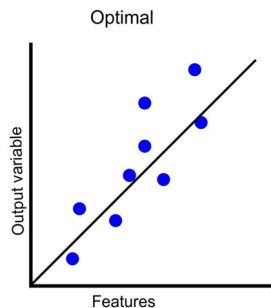
This is the section of the data set we use to train the model, our model might get really good at predicting this, but that does not mean the model works well

Testing data

The data we use to test how the model performs on unseen data. This gives a good idea of model robustness.

Overfitting

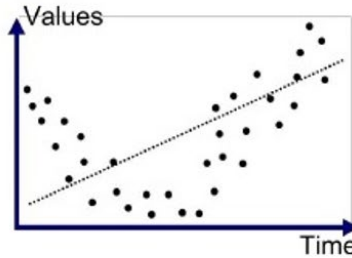
Overfitting occurs when a machine learning model learns the training data too well, capturing noise or random fluctuations in the data rather than the underlying patterns.



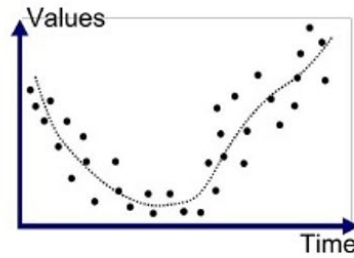
Under fitting

Model has not been trained enough

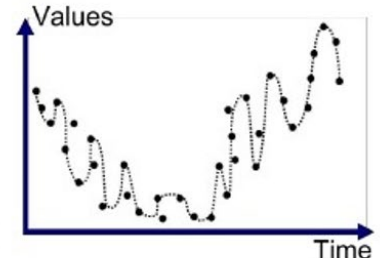
Data is not meaningful enough



Underfitted



Good Fit/Robust



Overfitted

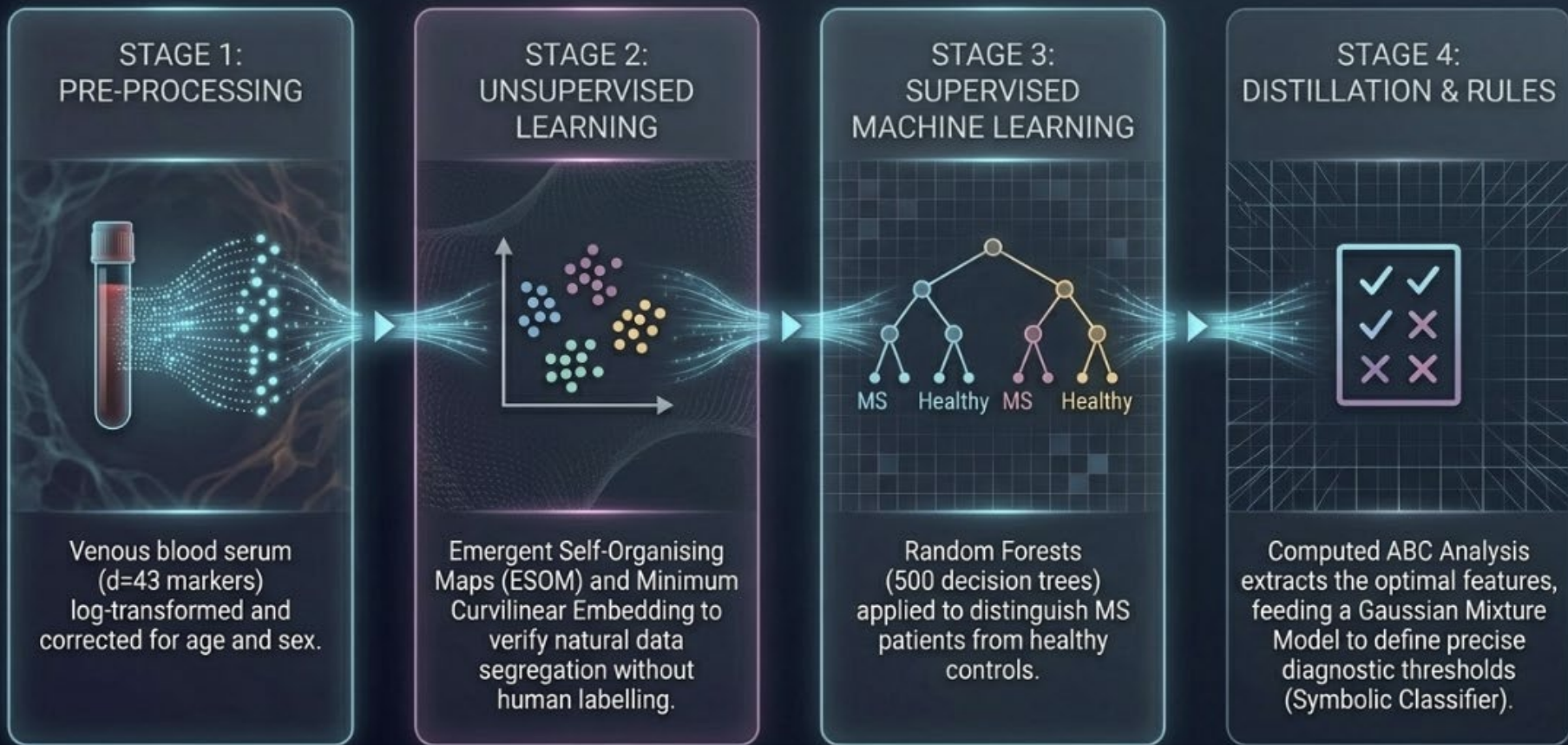
Real Life application of these tools

Article | [Open access](#) | Published: 05 October 2018

Machine-learning based lipid mediator serum concentration patterns allow identification of multiple sclerosis patients with high accuracy

[Jörn Lötsch](#) , [Susanne Schiffmann](#), [Katja Schmitz](#), [Robert Brunkhorst](#), [Florian Lerch](#), [Nerea Ferreiros](#),
[Sabine Wicker](#), [Irmgard Tegeder](#), [Gerd Geisslinger](#) & [Alfred Ultsch](#)

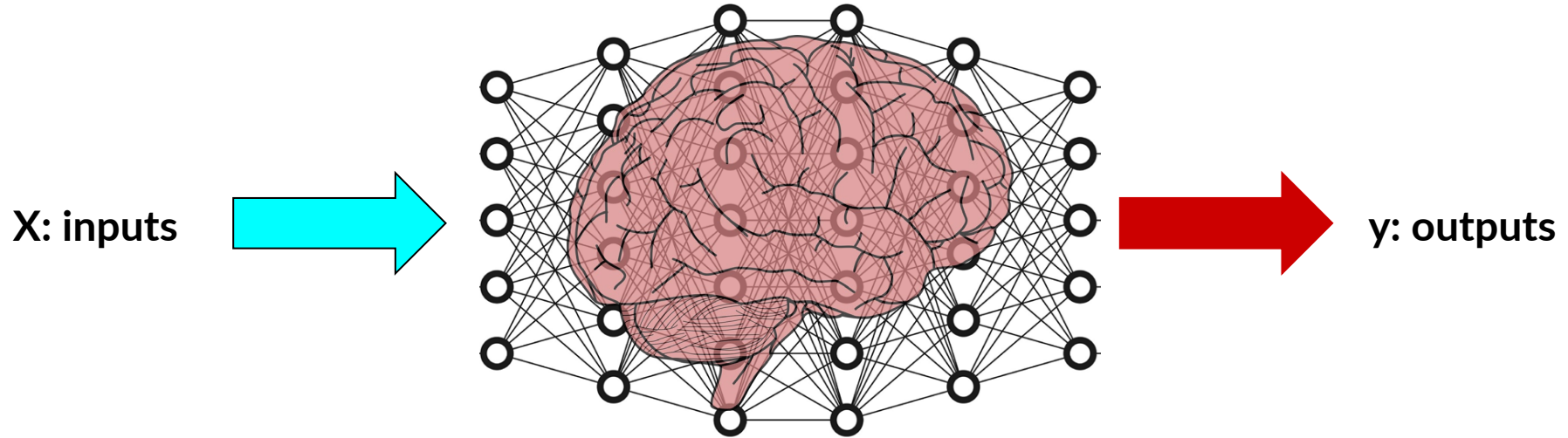
THE COMPUTATIONAL PIPELINE: FROM BLOOD SERUM TO DIAGNOSTIC RULE



Neural Networks

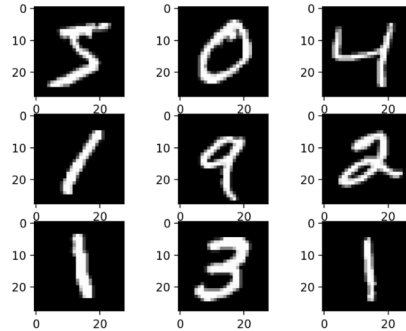


Neurons and Artificial Neural Networks



Artificial Neural Networks can be classifiers

Dataset

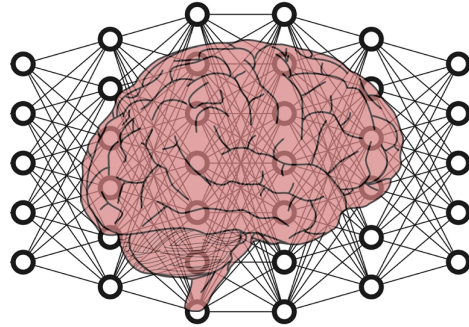
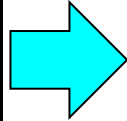
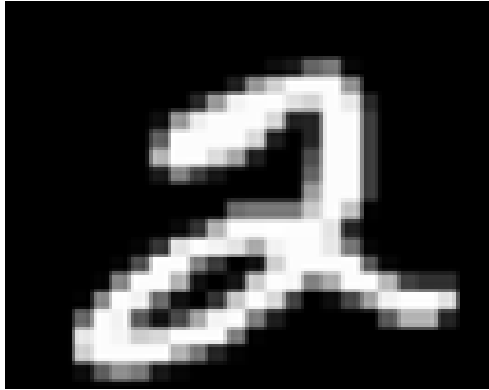


60,000 small square 28×28 pixel grayscale **images of handwritten single digits between 0 and 9**

Artificial Neural Networks can be classifiers

Image recognition

X:

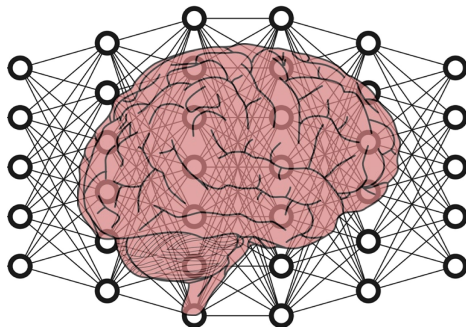
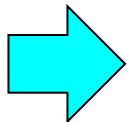
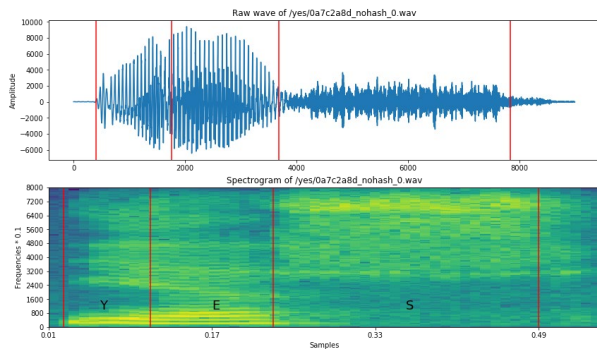


y: 2

Artificial Neural Networks can be classifiers

Speech recognition

X:



y: "yes"

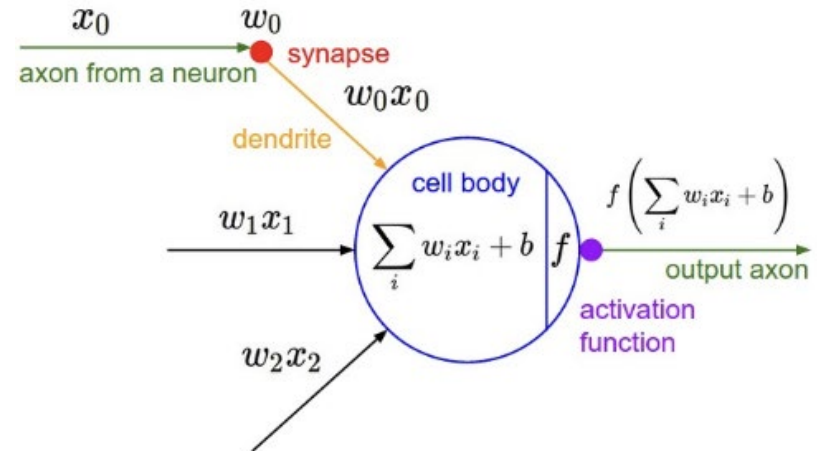
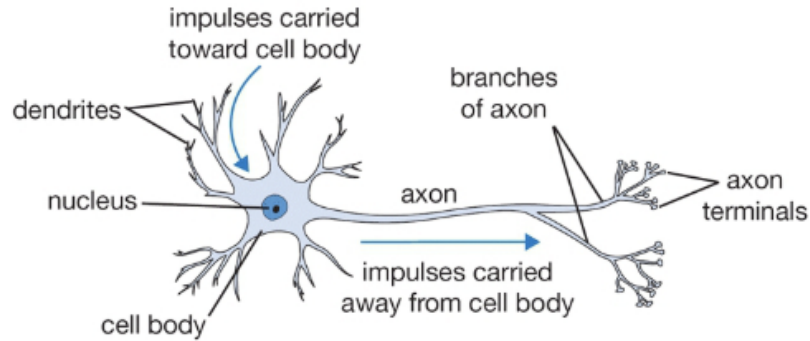
Spectrogram of the spoken word "yes"

Artificial Neural Networks

Can do countless things, including :

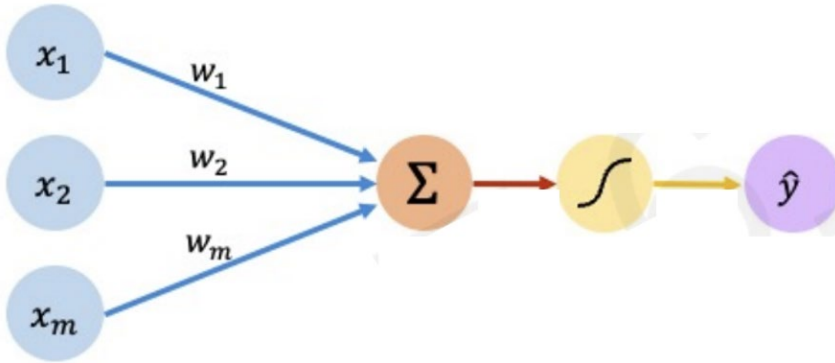
- **Natural Language Processing (NLP):**
Input: Text data (e.g., a sentence or document).
Output: Processed text (e.g., sentiment score, translated text, or named entities).
 - **Medical Diagnosis:**
Input: Medical images (e.g., X-rays, MRIs) or patient data (e.g., age, symptoms).
Output: Diagnosis or probability of a condition (e.g., presence of a tumor).
 - **Financial Services:**
Input: Historical financial data (e.g., stock prices, transaction records).
Output: Predictions (e.g., future stock prices, fraud detection alerts).
 - **Autonomous Vehicles:**
Input: Sensor data (e.g., camera images, LIDAR scans).
Output: Driving actions (e.g., steering angle, acceleration, braking).
 - **Recommendation Systems:**
Input: User behaviour data (e.g., past purchases, viewing history).
Output: Recommendations (e.g., movies, products)
-

Biological vs Artificial neuron



Perceptron: the singular neuron

The building block of Artificial Neural Networks



Inputs Weights Sum Non-Linearity Output

A diagram showing the mathematical formula for a perceptron output. The formula is $\hat{y} = g \left(\sum_{i=1}^m x_i w_i \right)$. A purple arrow points from the word "Output" to $\hat{y}$. A red arrow points from the text "Linear combination of inputs" to the summation term $\sum_{i=1}^m x_i w_i$. A yellow arrow points from the text "Non-linear activation function" to the function g .

Perceptron: the singular neuron

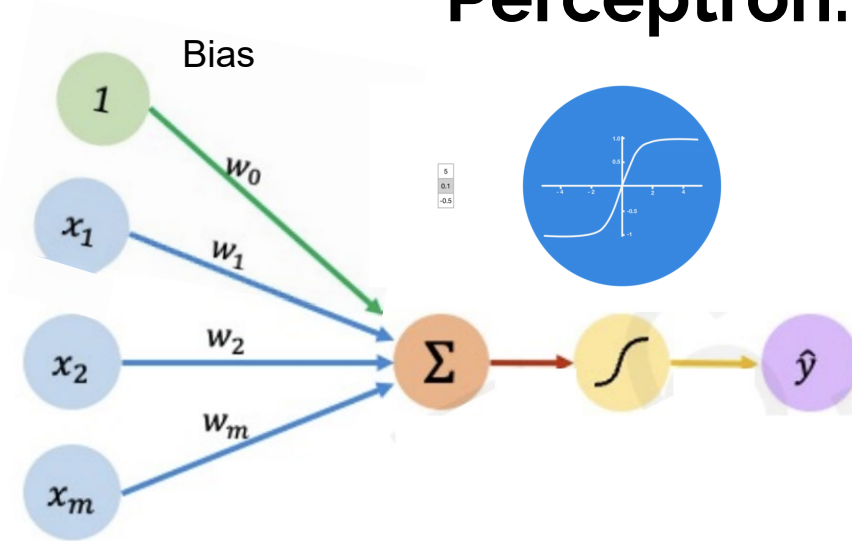


Diagram illustrating the mathematical representation of the perceptron output:

$$\hat{y} = g \left(w_0 + \sum_{i=1}^m x_i w_i \right)$$

Labels and arrows in the diagram:

- Output:** Points to $\hat{y}$ (purple arrow).
- Linear combination of inputs:** Points to the summation term $\sum_{i=1}^m x_i w_i$ (red arrow).
- Non-linear activation function:** Points to g (yellow arrow).
- Bias:** Points to w_0 (green arrow).

Inputs Weights Sum Non-Linearity Output

The bias term is a scalar that allows us to shift left or right along our activation function

Perceptron: the singular neuron

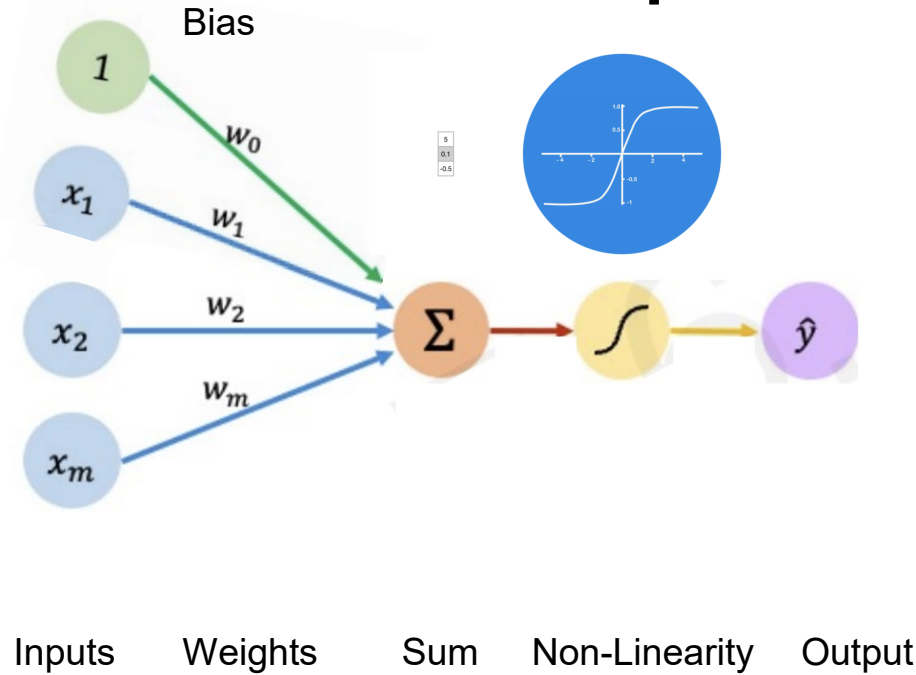


Diagram illustrating the mathematical representation of the perceptron output:

$$\hat{y} = g \left(w_0 + \sum_{i=1}^m x_i w_i \right)$$

Labels in the diagram:

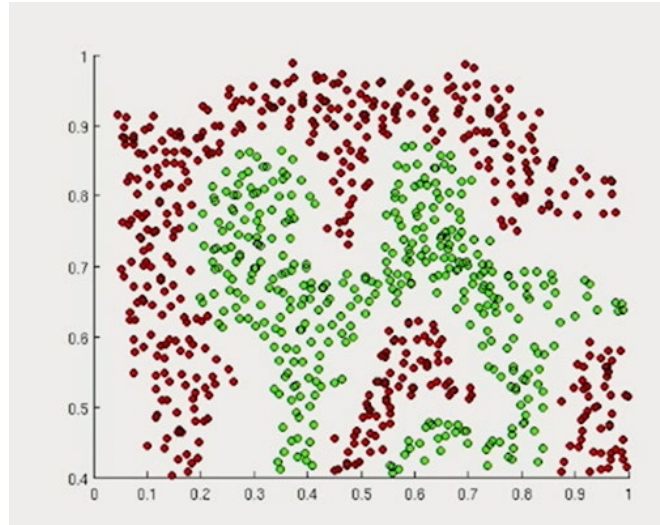
- Output: $\hat{y}$
- Linear combination of inputs: $w_0 + \sum_{i=1}^m x_i w_i$
- Non-linear activation function: g
- Bias: w_0

$$\hat{y} = g(w_0 + \mathbf{X}^T \mathbf{W})$$

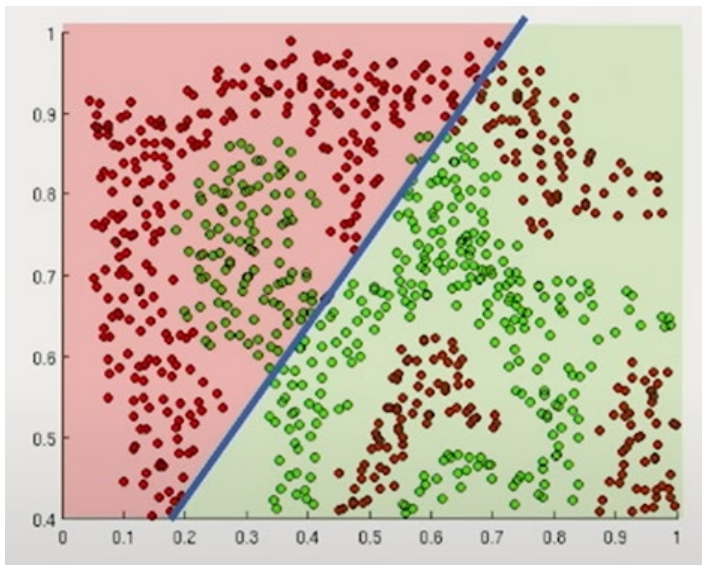
$$\text{where: } \mathbf{X} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix} \text{ and } \mathbf{W} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}$$

Why do we need Activation Functions?

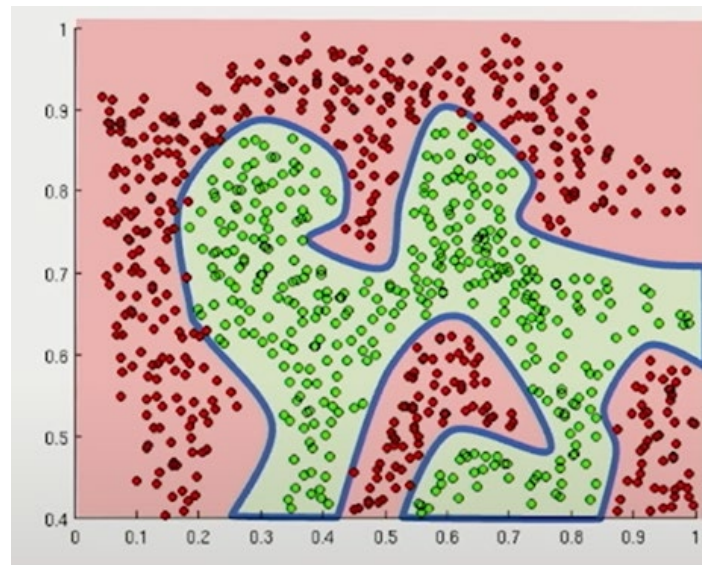
The purpose of activation functions is to introduce **non-linearities** into the network



Why do we need non-linearity?

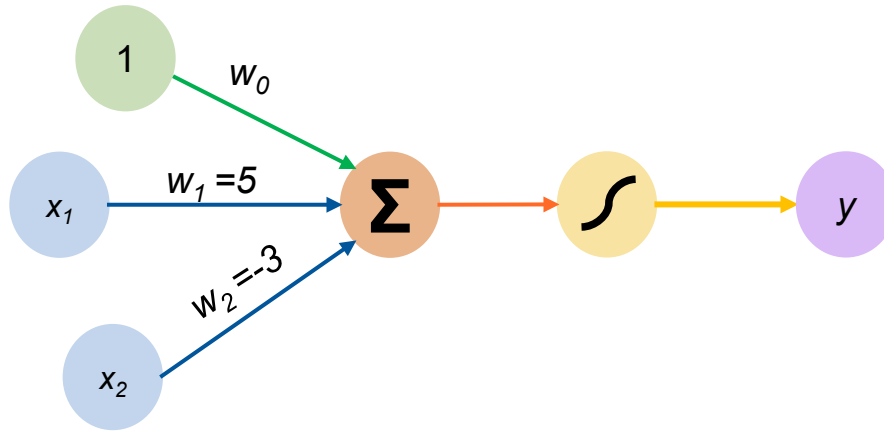


Linear activation functions produce **linear decisions** no matter the network size



Non-linearities allow us to **approximate** arbitrarily complex functions

Perceptron: an example



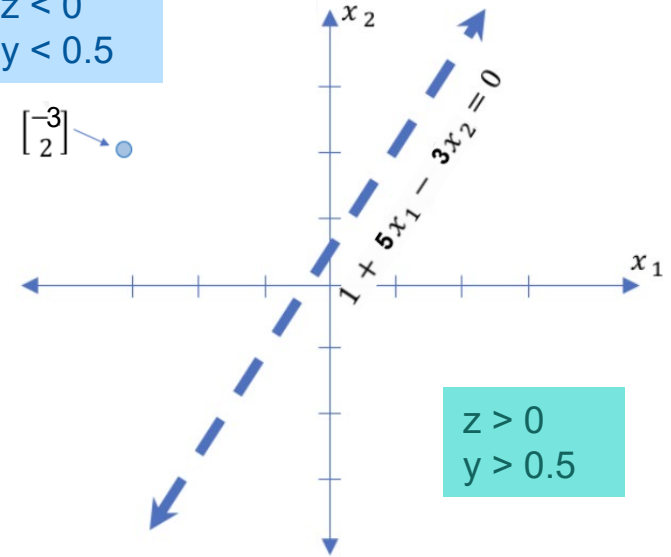
If we have an input $\mathbf{X} = \begin{bmatrix} -3 \\ 2 \end{bmatrix}$

$$\begin{aligned}\hat{y} &= g(1 + (5 \cdot -3) - (3 \cdot 2)) \\ &= g(-20) = 2.06 \text{ E-9}\end{aligned}$$

$$\hat{y} = g(\underbrace{1 + 5x_1 - 3x_2}_{(z)})$$

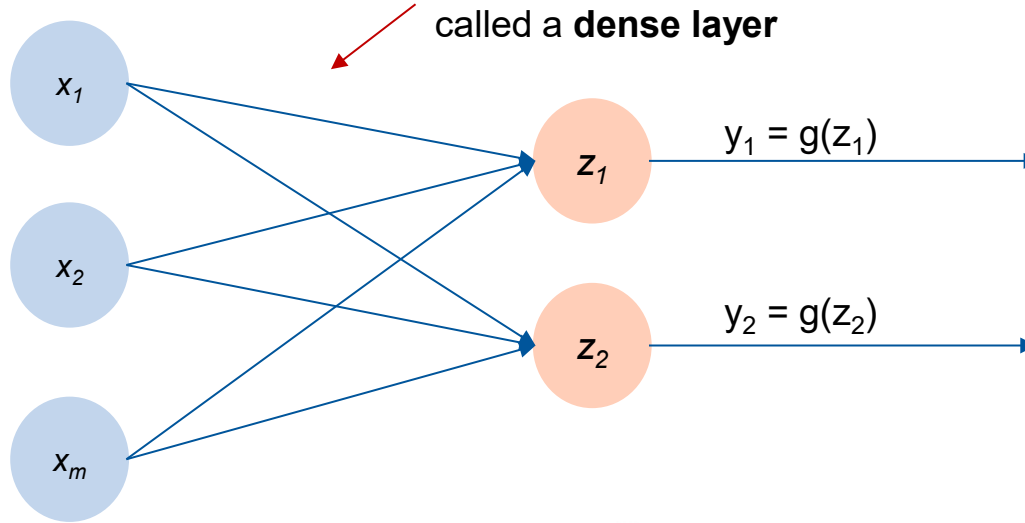
$$\begin{aligned}z &< 0 \\ y &< 0.5\end{aligned}$$

$$\begin{bmatrix} -3 \\ 2 \end{bmatrix}$$



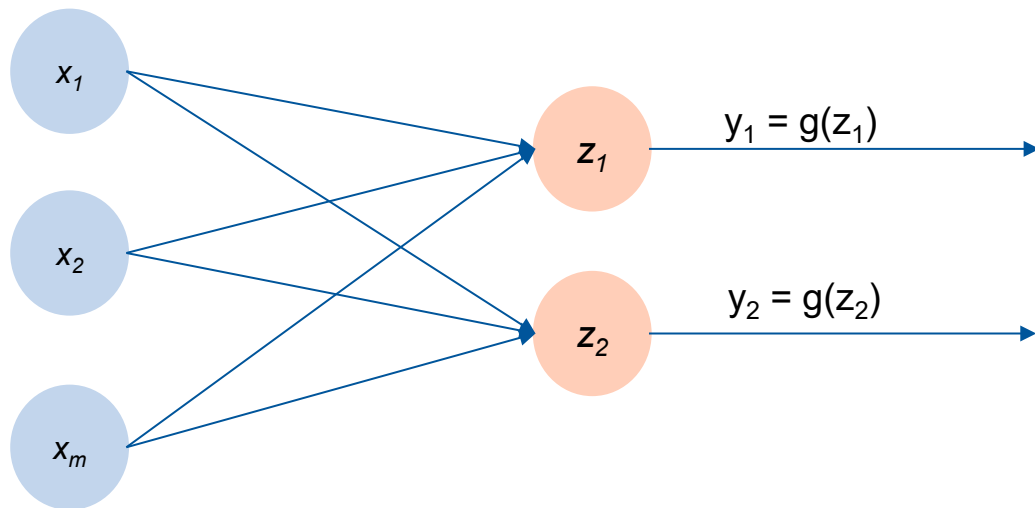
Multi-output perceptron

Because of the dense connections between inputs and outputs, this is called a **dense layer**



$$z_i = w_{0,i} + \sum_{j=1}^m x_j w_{j,i}$$

Multi-output perceptron

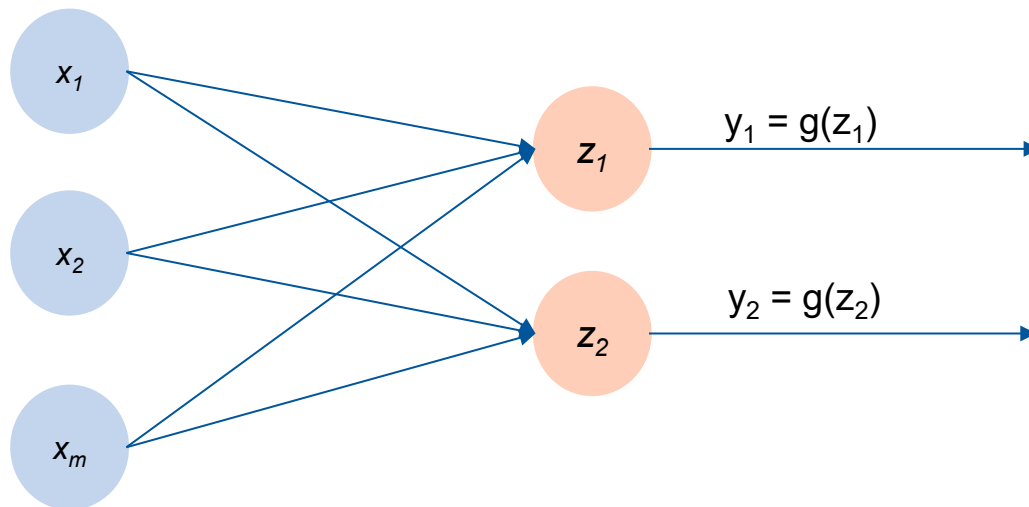


This is just an example to understand the intuition.

Multi-output perceptrons are useful when you need to predict multiple target variables simultaneously.

$$z_i = w_{0,i} + \sum_{j=1}^m x_j w_{j,i}$$

Multi-output perceptron



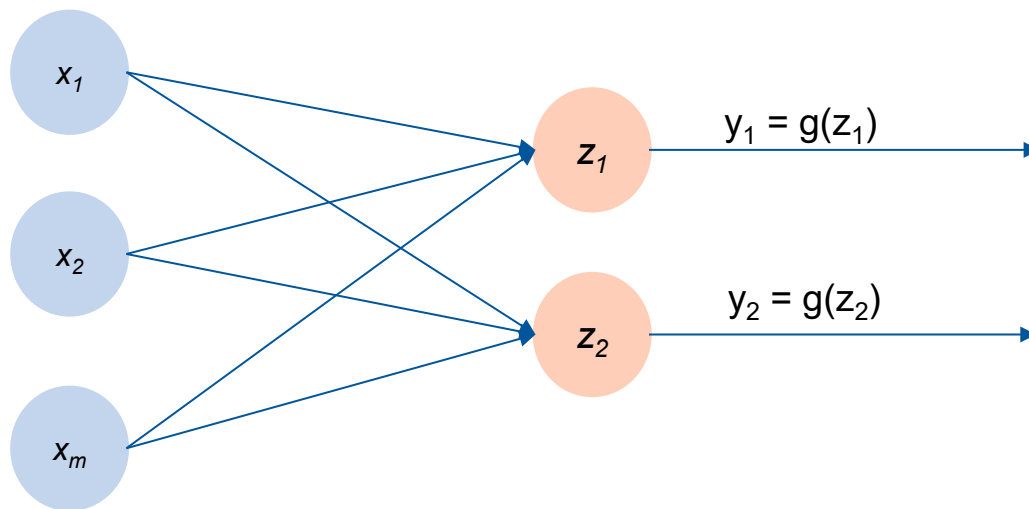
NEURAL DECODING

By inputting population neural activity (e.g. calcium imaging/spike trains) and output motor variables, decision outcomes, and predicted reward.

Multi-output advantage:
Simultaneous decoding of multiple behavioural signals components from the same neural state.

$$z_i = w_{0,i} + \sum_{j=1}^m x_j w_{j,i}$$

Multi-output perceptron



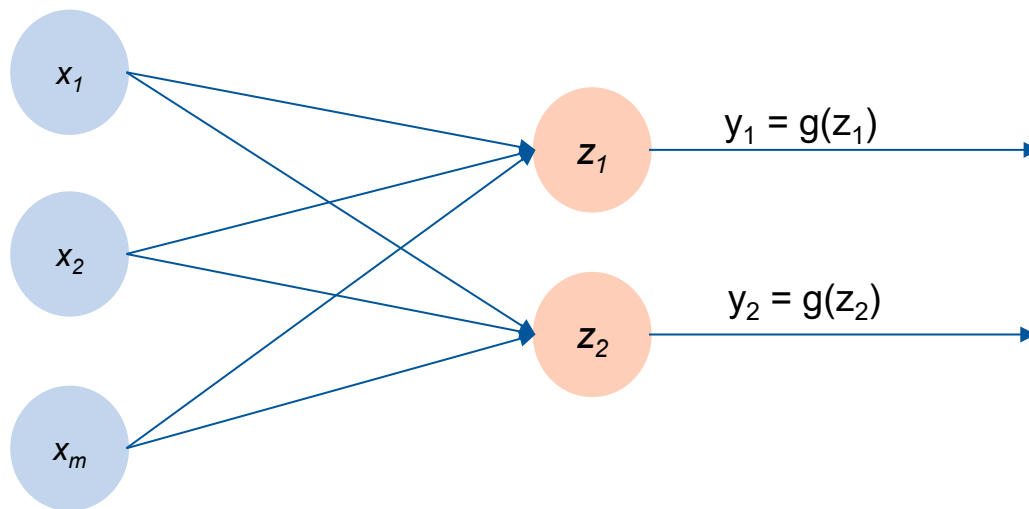
STIMULUS ENCODING

Like predicting neural responses to external stimuli, by inputting auditory/visual stimuli (e.g. frequency, orientation, temporal feature), and output the activity of multiple neurons.

Multi-output advantage: The network learns a population code, capturing how a group of neurons responds in parallel.

$$z_i = w_{0,i} + \sum_{j=1}^m x_j w_{j,i}$$

Multi-output perceptron



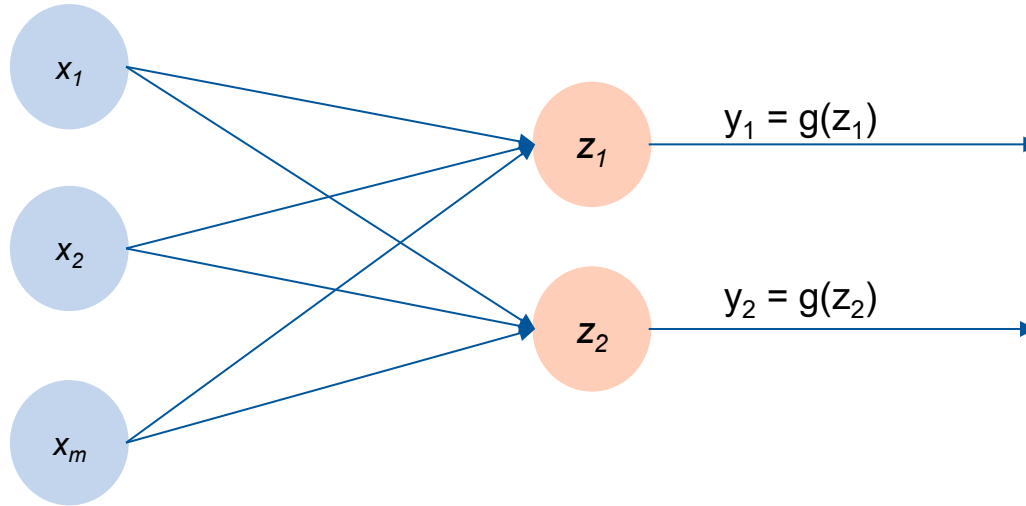
BEHAVIOURAL CLASSIFICATION WITH RICH LABELS

Like classifying trial types or internal states with multidimensional labels by predicting not only “lick” or “no lick” but also trial phase (stimulus, response), stimulus identity, etc.

Multi-output advantage: jointly learn latent task variables that may be entangled in the neural data.

$$z_i = w_{0,i} + \sum_{j=1}^m x_j w_{j,i}$$

Multi-output perceptron



```
from sklearn.neural_network import MLPRegressor
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split

# Generate synthetic data with multiple outputs
X, y = make_regression(n_samples=1000, n_features=20,
                      n_targets=2, noise=0.1)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2, random_state=42)

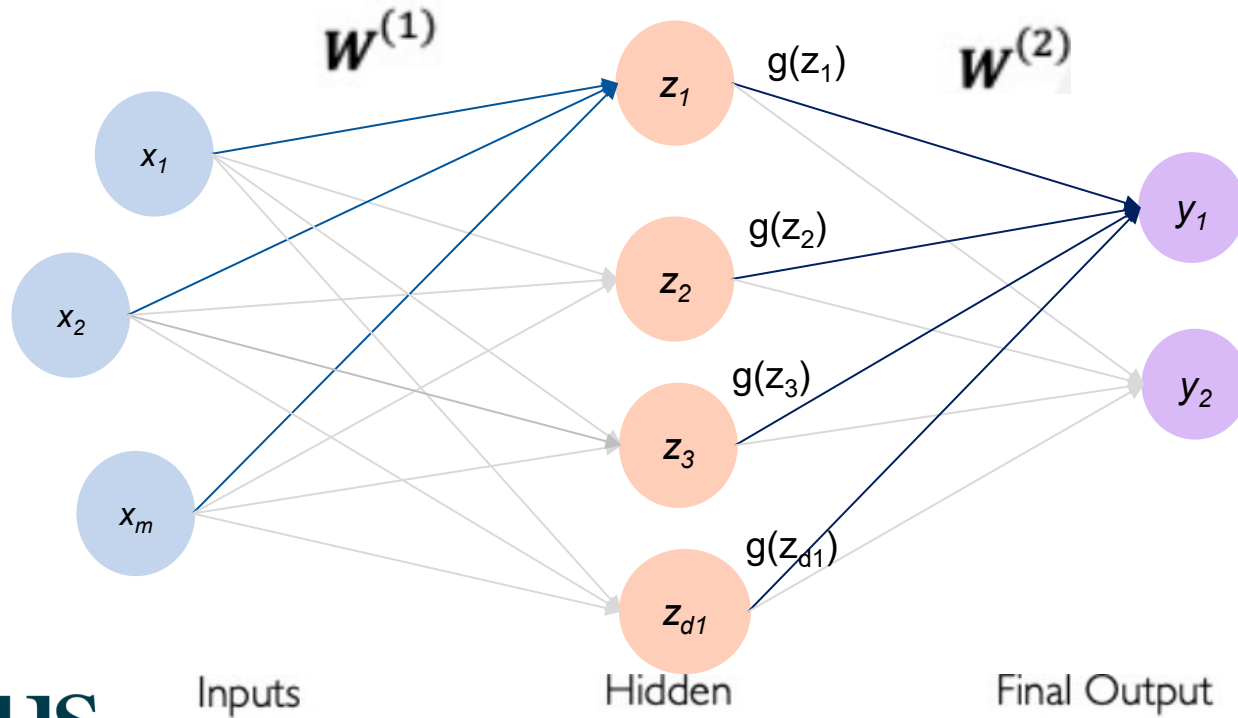
# Define the multi-output perceptron
mlp = MLPRegressor(hidden_layer_sizes=(100,),
                  activation='relu', max_iter=300)

# Train the neural network on the training data
mlp.fit(X_train, y_train)

# Make predictions on the test data
predictions = mlp.predict(X_test)
```

$$z_i = w_{0,i} + \sum_{j=1}^m x_j w_{j,i}$$

Single Layer Neural Network



In layered neural networks, the outputs from one layer become the inputs of the next layer

Single Layer Neural Network

For classification:

```
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

# Generate synthetic classification data
X, y = make_classification(n_samples=1000, n_features=20, n_classes=2,
random_state=42)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Define the single-layer neural network
clf = MLPClassifier(hidden_layer_sizes=(100,), activation='relu',
max_iter=300)

# Train the neural network on the training data
clf.fit(X_train, y_train)

# Make predictions on the test data
predictions = clf.predict(X_test)
```

For regression:

```
from sklearn.neural_network import MLPRegressor
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split

# Generate synthetic regression data
X, y = make_regression(n_samples=1000, n_features=20, noise=0.1,
random_state=42)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Define the single-layer neural network
reg = MLPRegressor(hidden_layer_sizes=(100,), activation='relu',
max_iter=300)

# Train the neural network on the training data
reg.fit(X_train, y_train)

# Make predictions on the test data
predictions = reg.predict(X_test)
```

Single Layer Neural Network

For classification:

```
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

# Generate synthetic classification data
X, y = make_classification(n_samples=1000, n_features=20, n_classes=2,
                          random_state=42)

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2, random_state=42)

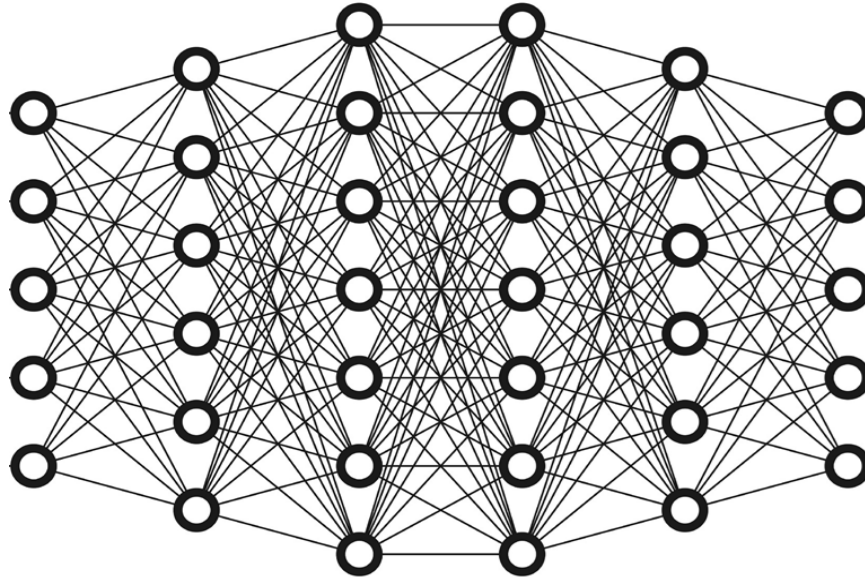
# Define the single-layer neural network
clf = MLPClassifier(hidden_layer_sizes=(100,), activation='relu',
                    max_iter=300)

# Train the neural network on the training data
clf.fit(X_train, y_train)

# Make predictions on the test data
predictions = clf.predict(X_test)
```

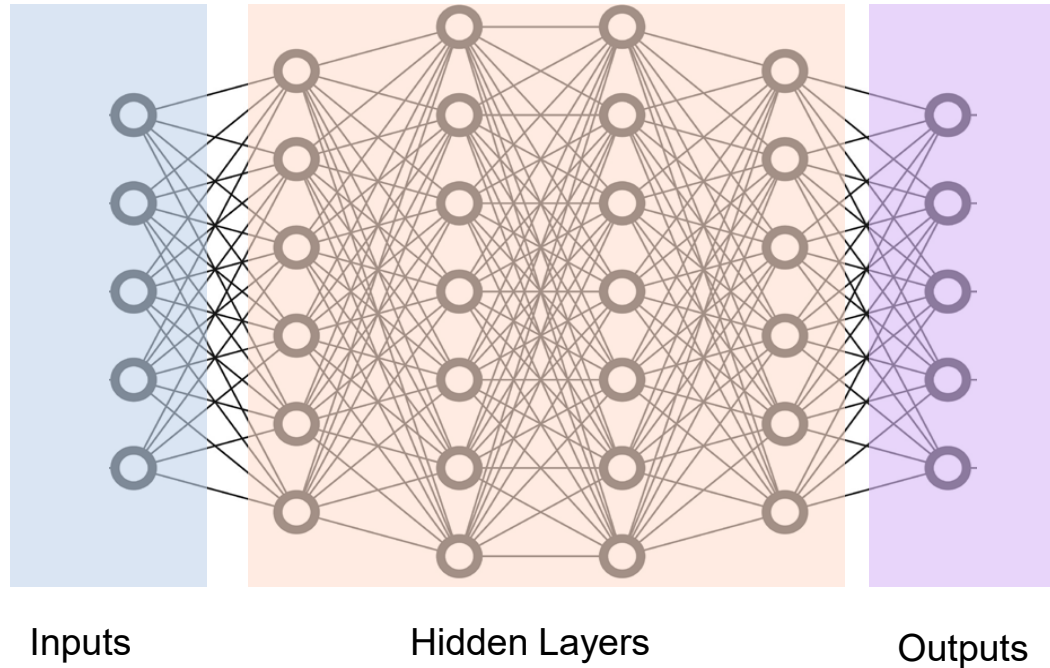
- **n_samples**= The number of samples (data points) in your dataset. 1000 creates 1000 data points.
- **n_features**=The number of features (input variables) in your dataset. 20 means each data point has 20 input variables.
- **n_classes**= It determines how many different categories the target variable (y) can take. 2 means the target variable (y) will have three distinct class labels (e.g., 0, 1).
- **random_state**=A seed value for random number generation to ensure reproducibility. 42 ensures the data generation process is reproducible.

Multi Layer Neural Network



In layered neural networks, the outputs from one layer become the inputs of the next layer

Multi Layer Neural Network



In layered neural networks, the outputs from one layer become the inputs of the next layer

Multi Layer Neural Network

For classification:

```
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

# Generate synthetic classification data
# n_samples: number of samples
# n_features: number of features
# n_classes: number of classes
# random_state: seed for reproducibility
X, y = make_classification(n_samples=1000, n_features=20, n_classes=2, random_state=42)

# Split the data into training and testing sets
# test_size: proportion of the dataset to include in the test split
# random_state: seed for reproducibility
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Define the multi-layer neural network with two hidden layers
# hidden_layer_sizes: tuple specifying the number of neurons in each hidden layer
# activation: activation function for the hidden layers ('relu' in this case)
# max_iter: maximum number of iterations for training
clf = MLPClassifier(hidden_layer_sizes=(100, 50), activation='relu', max_iter=300)

# Train the neural network on the training data
# X_train: training input data
# y_train: training target data
clf.fit(X_train, y_train)

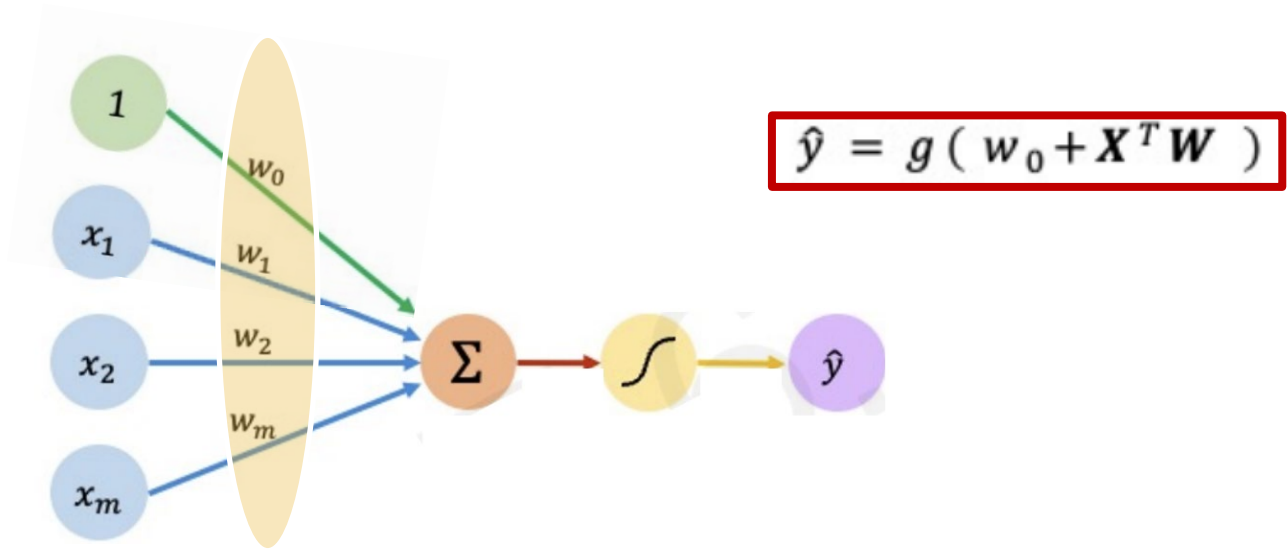
# Make predictions on the test data
# X_test: testing input data
predictions = clf.predict(X_test)
```

- **hidden_layer_sizes=** in this case it specifies two hidden layers, the first with 100 neurons and the second with 50 neurons.

Training a neural network

The code does it for us. But what does it do exactly?

Think about your data and think about this model



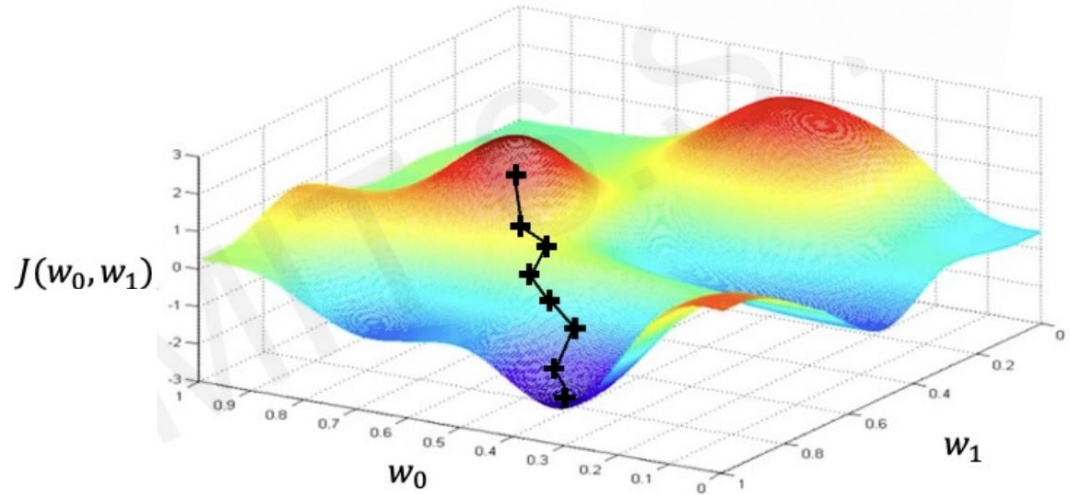
Is there any information we don't have?

Loss optimisation

- Quantify the Loss (how wrong the outputs y are compared to our test data)
- Update the weights (through gradient descent and backpropagation) to minimise this loss

Loss optimisation

- Quantify the Loss (how wrong the outputs y are compared to our test data)
- Update the weights (through gradient descent and backpropagation) to minimise this loss



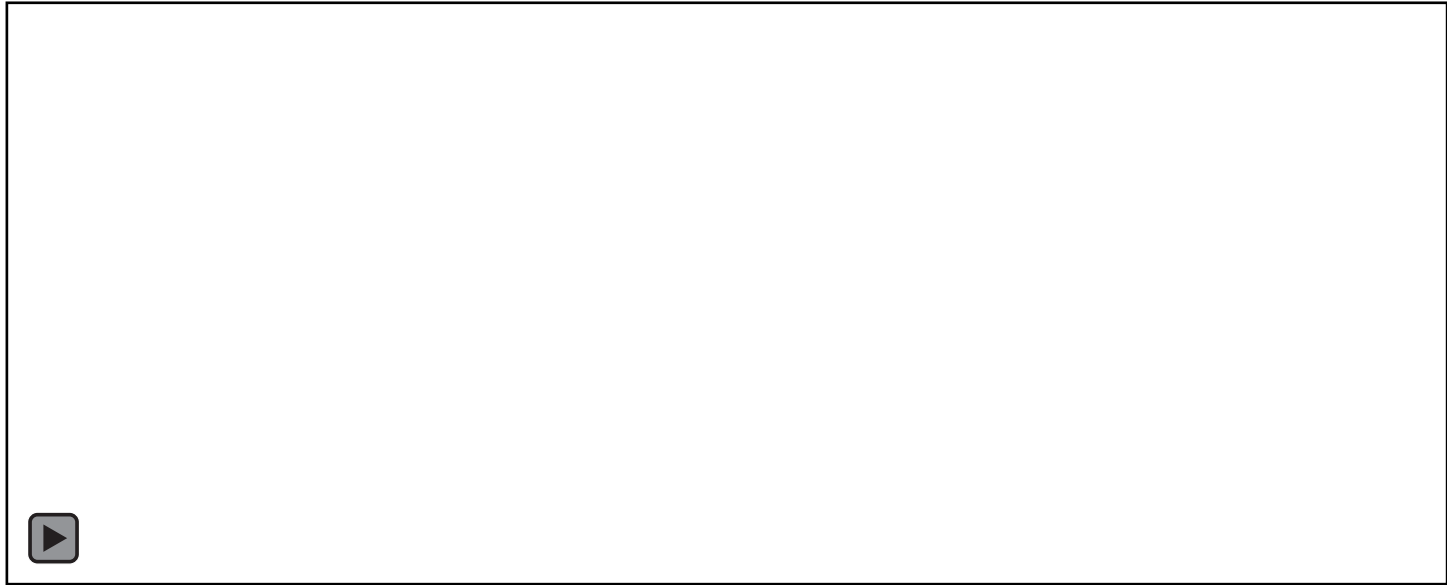
scikit-learn handles all the underlying details of backpropagation and gradient descent, allowing you to focus on defining and training your model

The code handles this for us

Types of ANNs

Convolutional Neural Networks (CNNs)

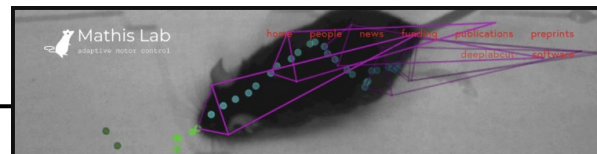
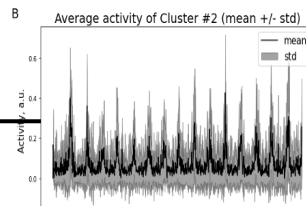
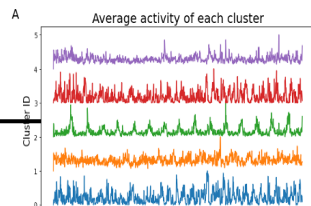
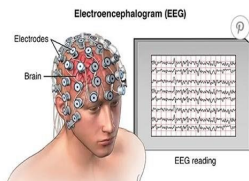
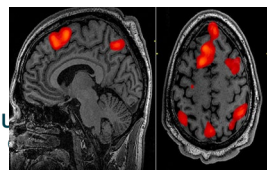
CNNs are designed for visual data processing.



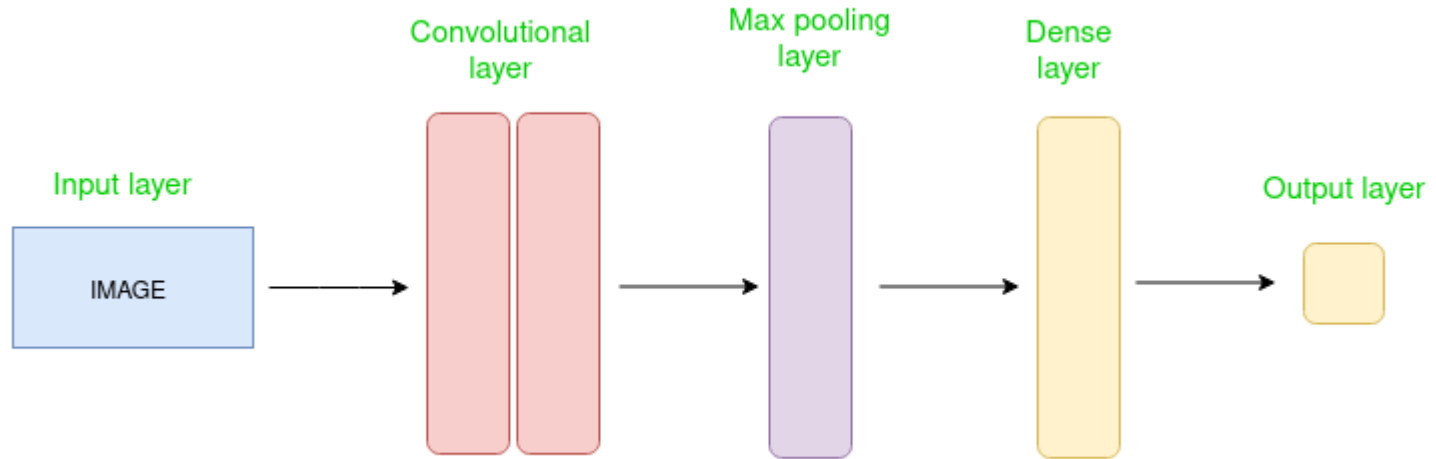
Convolutional neural networks in neuroscience

- **Brain Imaging:**
 - Structural MRI/fMRI: Disease diagnosis, brain age prediction
 - Calcium imaging: Spike deconvolution, neuron segmentation
- **Signal Processing:**
 - Spike sorting from extracellular recordings
 - EEG/LFP decoding of brain states
- **Behavioural Analysis:**
 - Pose estimation and action recognition from video (e.g., DeepLabCut)
- **Computational Neuroscience:**
 - Modelling visual cortex responses to natural stimuli
 - Feature extraction and representational similarity analysis

Image Suggestion: Grid of application icons (brain scan, electrode trace, video frame, CNN-visual cortex overlay)



Convolutional neural networks



Convolutional neural networks

Input layer

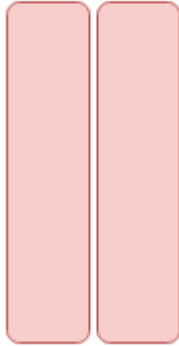


Let's take an example by running an image of dimension $32 \times 32 \times 3$.

- **Input Layers:** It's the layer in which we give **input to our model**. In CNN, Generally, the input will be an image or a sequence of images. This layer holds the raw input of the image with width 32, height 32, and depth 3 (= the colour array).

Convolutional neural networks

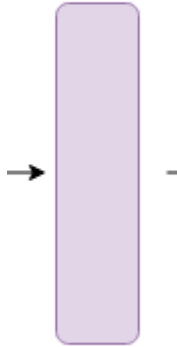
Convolutional
layer



- **Filters/Kernels:** Small matrices that slide over the input image. *Each filter is designed to detect specific features like edges, textures, or patterns.*
- **Convolution Operation:** The filter is applied to the input image, producing a feature map. This operation involves multiplying the filter values with the corresponding pixel values and summing them up. The result is a new matrix that highlights the presence of the feature detected by the filter. *As the window slides over the image, it creates a new picture (feature map) that highlights where it found those edges or patterns.*
- **ReLU:** After the window slides over the image, ReLU changes all negative values to zero. *This helps the network focus on important features and ignore less useful information.*

Convolutional neural networks

Max pooling
layer



- **Max Pooling:** This layer reduces the dimensionality of the feature map while retaining important features. It does this by selecting the maximum value from a small region (e.g., 2x2) of the feature map. This helps in reducing computational load and preventing overfitting.
- *Think of s zooming out on a picture to make it smaller while keeping the important parts. Max pooling picks the biggest value from a small area, reducing the size of the feature map but keeping the key features.*

Convolutional neural networks

Dense
layer



- **Flattening:** Converts the 2D feature maps into a 1D vector. This step prepares the data for the fully connected layers. *Imagine taking all the important features from the picture and laying them out in a single line.*
- **Dense Layers:** These layers connect every neuron from one layer to every neuron in the next layer. They combine the features learned by the convolutional and pooling layers to make final predictions. *This is where the network decides which class the input image belongs to.*

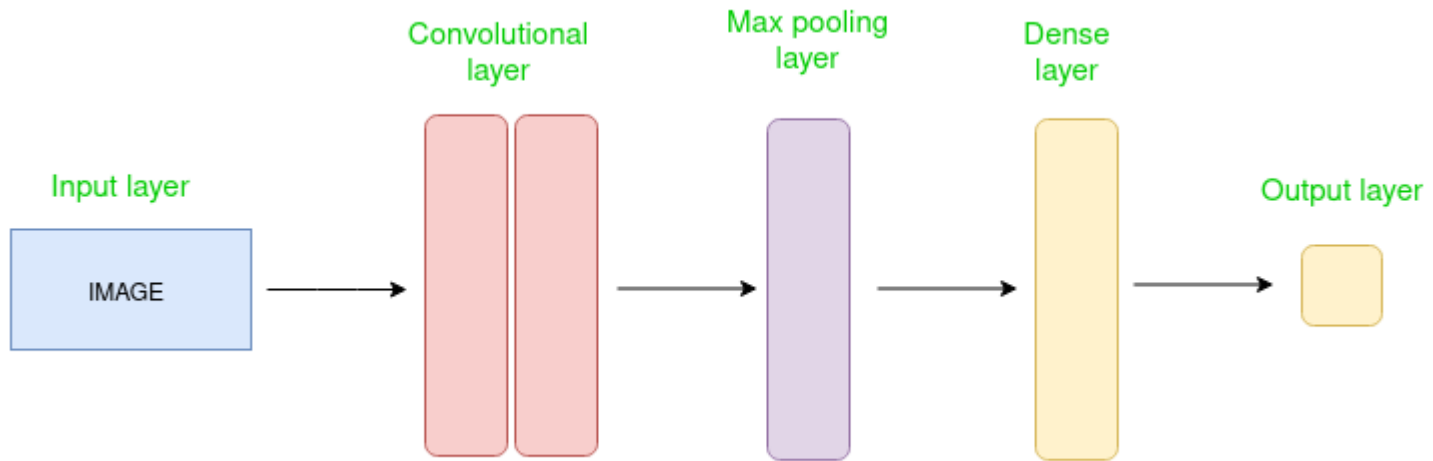
Convolutional neural networks

Output layer



- **Output Layer:** The output from the fully connected layers is then fed into a logistic function for classification tasks like *sigmoid* or *softmax* which converts the *output of each class into the probability score of each class*

Convolutional neural networks



Convolutional neural networks

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

# Define the CNN model
model = Sequential()

# Add a convolutional layer with 32 filters, a 3x3 kernel, and ReLU activation
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(64, 64, 3)))

# Add a max pooling layer with a 2x2 pool size
model.add(MaxPooling2D(pool_size=(2, 2)))

# Add another convolutional layer with 64 filters
model.add(Conv2D(64, (3, 3), activation='relu'))

# Add another max pooling layer
model.add(MaxPooling2D(pool_size=(2, 2)))

# Flatten the feature maps to a 1D vector
model.add(Flatten())

# Add a fully connected layer with 128 neurons and ReLU activation
model.add(Dense(128, activation='relu'))

# Add the output layer with softmax activation for multi-class classification
model.add(Dense(10, activation='softmax'))

# Compile the model with Adam optimizer and categorical crossentropy loss
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Print the model summary
model.summary()
```

1.Convolutional Layers:

1. **Number of Filters:** The choice of 32 and 64 filters is typical for initial layers in a CNN. These numbers can be adjusted based on the complexity of the task and the size of the dataset.
2. **Kernel Size:** A 3x3 kernel is a standard choice that balances computational efficiency and the ability to capture spatial features.

2.Pooling Layers:

1. **Pool Size:** A 2x2 pool size is commonly used to reduce the spatial dimensions while retaining important features.

3.Fully Connected Layers:

1. **Number of Neurons:** The choice of 128 neurons in the dense layer is a common practice, but it can be tuned based on the complexity of the task.

4.Activation Functions:

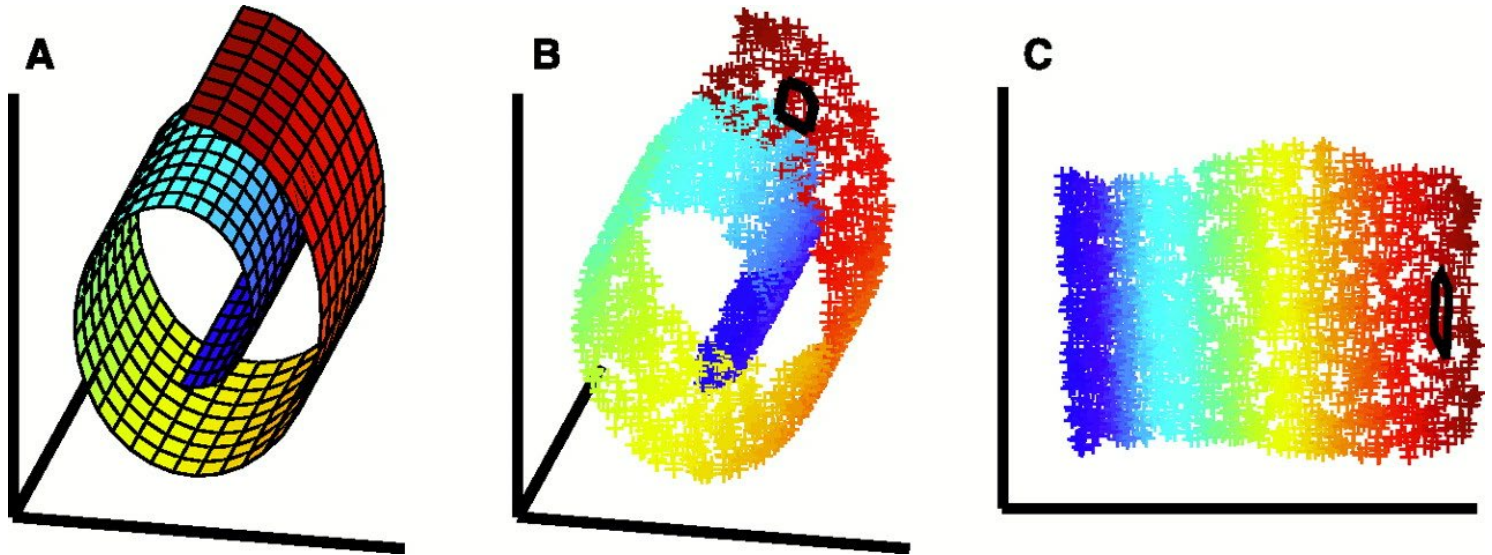
1. **ReLU:** The ReLU activation function is widely used due to its ability to mitigate the vanishing gradient problem and speed up training.
2. **Softmax:** Softmax is used in the output layer for multi-class classification tasks.

5.Optimizer and Loss Function:

1. **Adam Optimizer:** Adam is a popular choice for its adaptive learning rate and efficiency.
2. **Categorical Crossentropy:** This loss function is suitable for multi-class classification problems.

EXTRA INFO IF YOU'RE CURIOUS

Non-Linear Dimensionality Reduction



Manifolds

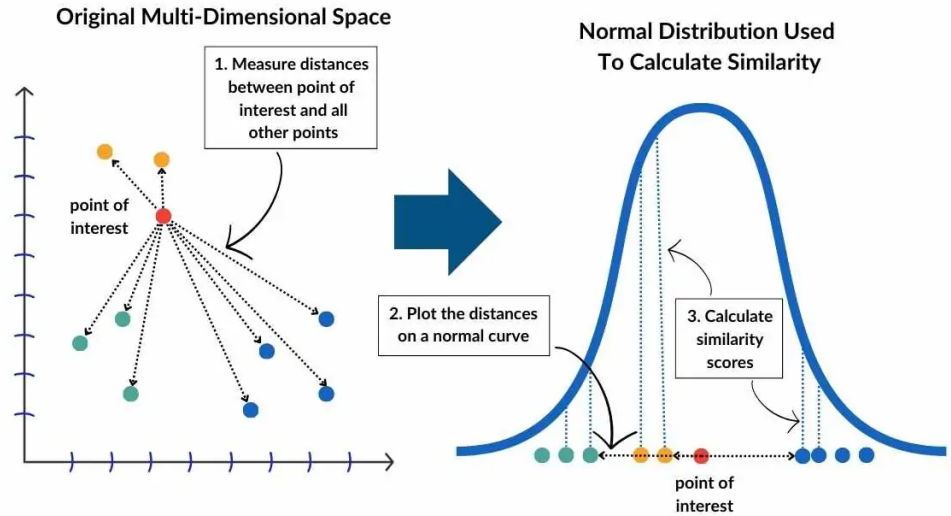


t-SNE

t-SNE is a nonlinear dimensionality reduction method used mainly for visualizing high-dimensional data in 2D or 3D.

It focuses on preserving local structure, meaning it keeps similar data points close together after reducing dimensions.

Unlike PCA, t-SNE does not create new axes as linear combinations. Instead, it builds a map that reflects the relative similarity between points, even if the original relationships are complex or nonlinear.



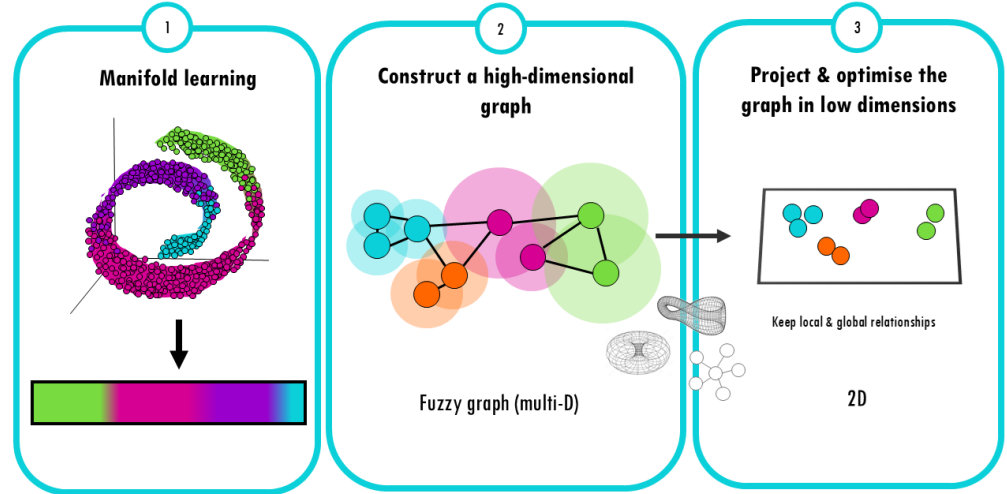
UMap

UMAP is a nonlinear dimensionality reduction method that, like t-SNE, is used to visualize high-dimensional data in 2D or 3D.

It aims to preserve both local and some global structure, making it better at maintaining the overall shape of the data.

Unlike PCA (linear) and t-SNE (local focus), UMAP builds a graph of neighborhood relationships and finds a low-dimensional space that best preserves those.

How does UMAP summarise many dimensions into 2?



Choosing a method

<i>Method</i>	<i>Type</i>	<i>Output</i>	<i>Best for</i>
PCA	Linear	Axes of variance	Denoising, compression
t-SNE	Nonlinear	3d/2d embeddings	Visualisation of local clusters
UMAP	Nonlinear	3d/2d embeddings	Visualisation + structural relationships
VAE	Deep learning, nonlinear	Low-dim latent space + decoder	Modelling, simulation, generative tasks

Problem 1:

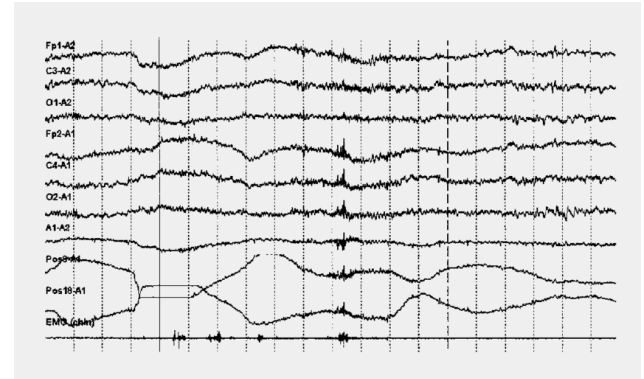
Removing artifacts from EEG recordings

You have collected EEG data for studying attention during a visual task. However, EEG signals have artifacts (e.g., eye movement) you need to clean before analysis. What method should you use?



Answer:

PCA, because it is a linear method that separates signals based on their variance. Since artifacts like eye blinks cause large variance, PCA can isolate and remove them, helping clean EEG data before analysis.



Problem 2:

Calcium Imaging – Find Brain States

You are studying hippocampal neural activity in mice during spatial navigation. You record from hundreds of neurons over multiple sessions and want to find a low-dimensional representation of the neural population that captures the temporal dynamics of place cell activity. Additionally, you want to reconstruct and generate realistic neural activity patterns to understand how spatial memories are encoded and replayed.

What method should you use?

Answer:

UMAP, because neural activity during decision-making is nonlinear and complex, and UMAP preserves both local clusters (to distinguish similar brain states) and global structure (to understand how these states relate across task phases like cue, decision, and movement), allowing you to clearly identify distinct brain states throughout the task.



Problem 2:

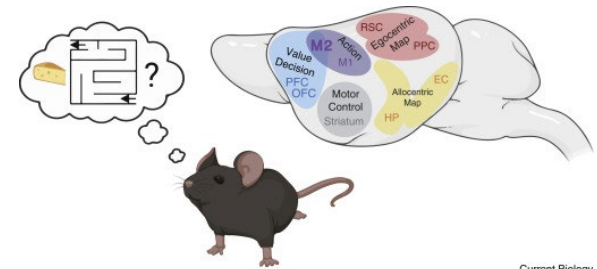
Spatial Navigation over Time

You are studying how the hippocampus encodes spatial memories in mice during maze navigation. You record from hundreds of neurons over time and want to reduce this complex activity into a low-dimensional space that captures the temporal dynamics of neural firing. You also want to reconstruct or simulate neural activity patterns to study memory replay.

What method should you use?

Answer:

VAE, because it learns a nonlinear, low-dimensional latent space that captures both the structure and variability of neural data. It allows for dimensionality reduction, while also providing a decoder to reconstruct or generate realistic activity patterns.

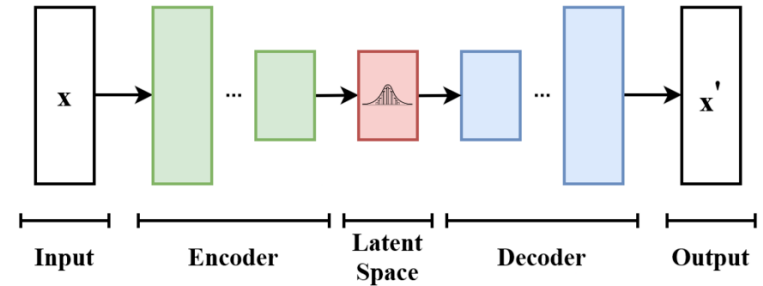


Current Biology

Variational Autoencoders (VAEs)

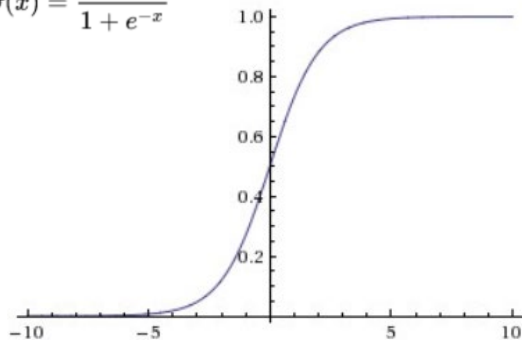
A VAE is a deep learning-based nonlinear method for dimensionality reduction and generative modeling.

It learns to compress data into a low-dimensional latent space and then reconstruct it from that space - guided by probabilistic encoding (outputs are distributions, not fixed points).



What types of activation functions are there and how do we pick one?

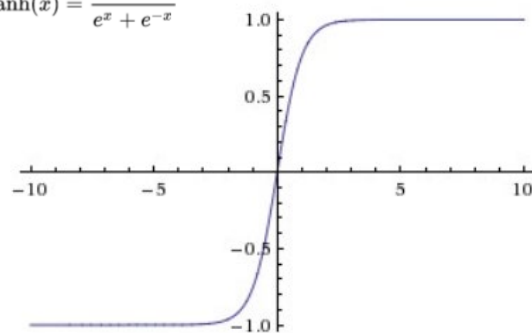
$$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$$



Sigmoid non-linearity squashes the numbers to range between $[0, 1]$.

Used for models where we have to predict the probability as an output

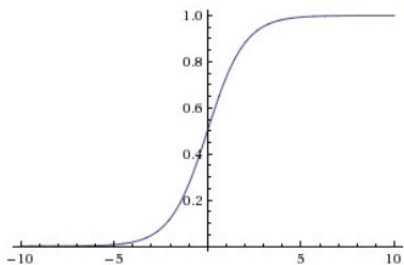
$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



tanh non-linearity squashes the numbers to range between $[-1, 1]$

Tanh tends to be preferred to sigmoid

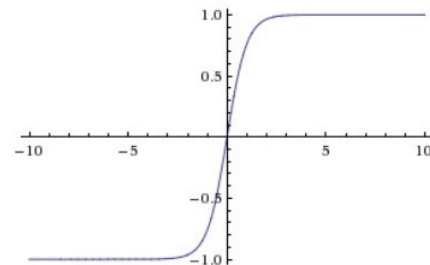
What types of activation functions are there and how do we pick one?



```
from sklearn.neural_network import MLPClassifier

# Define the neural network with the sigmoid activation
function
clf = MLPClassifier(activation='logistic', hidden_layer_sizes=
(100,), max_iter=300)

# Train the neural network on the training data
clf.fit(X_train, y_train)
```

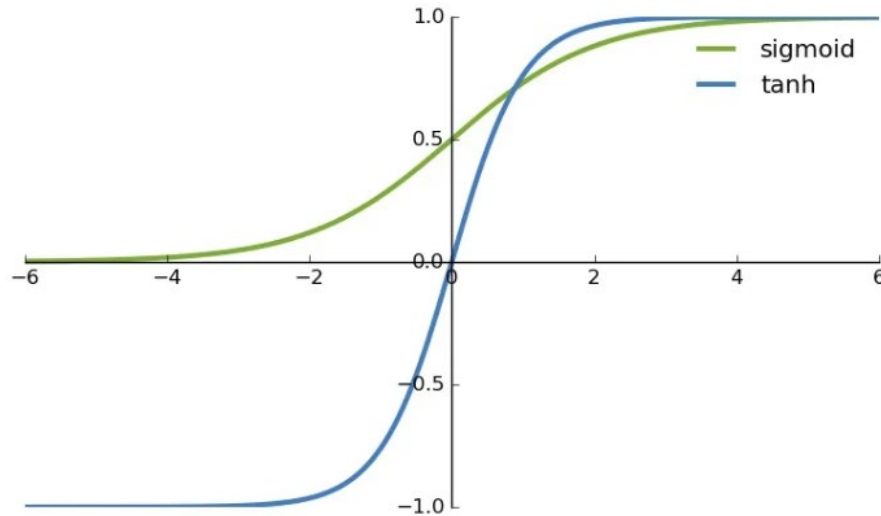


```
from sklearn.neural_network import MLPClassifier

# Define the neural network with the tanh activation function
clf = MLPClassifier(activation='tanh', hidden_layer_sizes=
(100,), max_iter=300)

# Train the neural network on the training data
clf.fit(X_train, y_train)
```

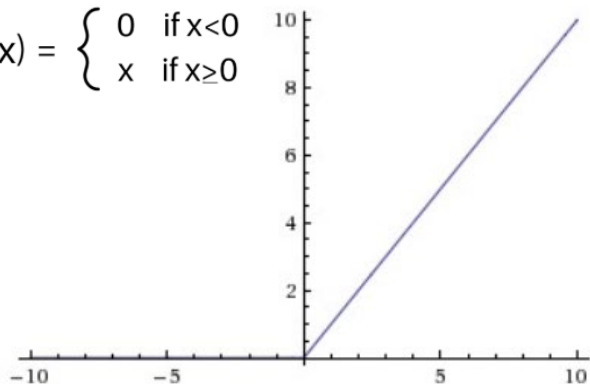
What types of activation functions are there and how do we pick one?



What types of activation functions are there and how do we pick one?

$$f(x) = \max(0, x)$$

$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$

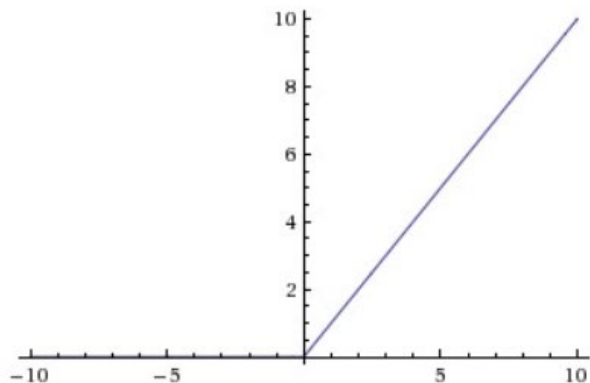


ReLU (Rectified Linear Unit) activation function is zero when $x < 0$ and then linear with slope 1 when $x > 0$

- **ReLU speeds up** the calculations up to 6x compared to **tanh**
- **ReLU units** can be fragile during training and can “die”. (i.e. neurons that never activate across the entire training dataset) if the learning rate is set too high. **With a proper setting of the learning rate this is less frequently an issue.**

What types of activation functions are there and how do we pick one?

$$f(x) = \max(0, x)$$



ReLU (Rectified Linear Unit) activation function is zero when $x < 0$ and then linear with slope 1 when $x > 0$

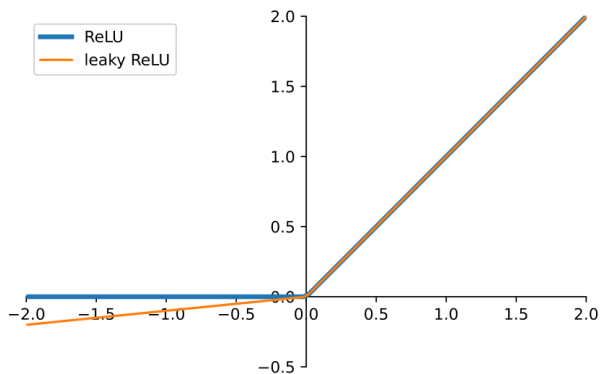
```
from sklearn.neural_network import MLPClassifier

# Define the neural network with the ReLU activation function
clf = MLPClassifier(activation='relu', hidden_layer_sizes=(100,), max_iter=300)

# Train the neural network on the training data
clf.fit(X_train, y_train)
```

What types of activation functions are there and how do we pick one?

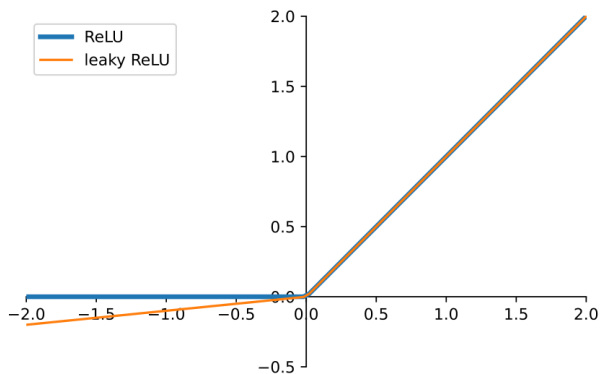
$$f(x) = \begin{cases} 0.01x & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$



- **Leaky ReLUs** are one attempt to fix the “dying ReLU” problem.
- Instead of the function being zero when $x < 0$, a leaky ReLU will instead have a small positive slope (of 0.01, or so). That is, the function computes $f(x) = 1(x < 0)(\alpha x) + 1(x \geq 0)(x)$ where α is a small constant.

Other solutions to the “dying ReLU” problem include ELUs (Exponential Linear Units) and PReLUs (Parametric ReLU).

What types of activation functions are there and how do we pick one?



- **Leaky ReLUs** are one attempt to fix the “dying ReLU” problem.

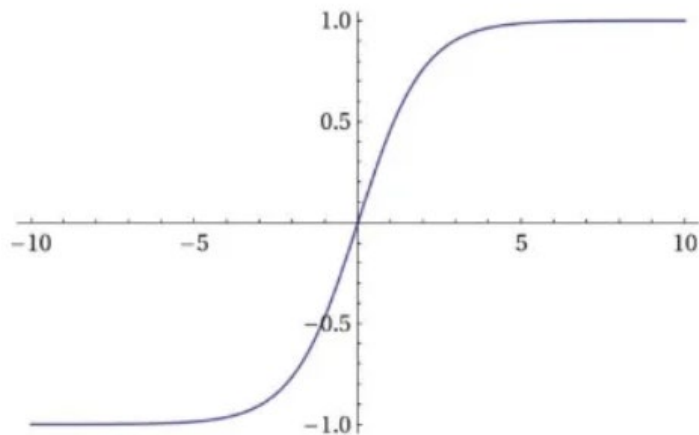
```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LeakyReLU

# Define the neural network model
model = Sequential([
    # Add a dense layer with 100 neurons
    Dense(100, input_dim=input_dim),
    # Apply Leaky ReLU activation function with alpha=0.01
    LeakyReLU(alpha=0.01),
    # Add the output layer with softmax activation for multi-class classification
    Dense(output_dim, activation='softmax')
])

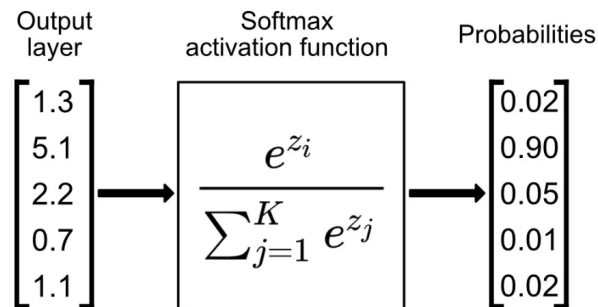
# Compile the model with Adam optimizer and categorical crossentropy loss
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Train the neural network on the training data
model.fit(X_train, y_train, epochs=300)
```

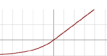
What types of activation functions are there and how do we pick one?



- **Softmax** is used when you need to classify data into more than two categories. For example, if you have a dataset with images of cats, dogs, and birds, Softmax can help determine the probability that a given image belongs to each of these classes.
- It is applied after the output.



What types of activation functions are there and how do we pick one?

ACTIVATION FUNCTION	PLOT	EQUATION	DERIVATIVE	RANGE
Linear		$f(x) = x$	$f'(x) = 1$	$(-\infty, \infty)$
Binary Step		$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{if } x \neq 0 \\ \text{undefined} & \text{if } x = 0 \end{cases}$	$\{0, 1\}$
Sigmoid		$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = f(x)(1 - f(x))$	$(0, 1)$
Hyperbolic Tangent(tanh)		$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	$f'(x) = 1 - f(x)^2$	$(-1, 1)$
Rectified Linear Unit(ReLU)		$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x > 0 \\ \text{undefined} & \text{if } x = 0 \end{cases}$	$[0, \infty)$
Softplus		$f(x) = \ln(1 + e^x)$	$f'(x) = \frac{1}{1 + e^{-x}}$	$(0, 1)$
Leaky ReLU		$f(x) = \begin{cases} 0.01x & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0.01 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$	$(-1, 1)$
Exponential Linear Unit(ELU)		$f(x) = \begin{cases} \alpha(e^x - 1) & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases}$	$f'(x) = \begin{cases} \alpha e^x & \text{if } x < 0 \\ 1 & \text{if } x > 0 \\ 1 & \text{if } x = 0 \text{ and } \alpha = 1 \end{cases}$	$[0, \infty)$